# MÓDULO PROYECTO

CURSO 2025__- 2026__

TITULO: MARYSHOPPING

ALUMNO: MARISOL VARELA CABADO

CICLO FORMATIVO: DESARROLLO DE APLICACIONES WEB(DAW)

TUTOR: BELKIS DÍAZ LAZURICA

# RESUMEN

El proyecto consiste en el desarrollo completo de una aplicación de comercio electrónico llamada MarySHopping. El proyecto se organiza como un monorepo con tres carpetas:

- Backend REST implementado con Java 25 y Spring Boot 4.1.0-M2.
- Frontend desarrollado con Nuxt 4 (Vue 3), Vuetify 4, Pinia, TypeScript e i18n (internacionalización).
- Una infraestructura de despliegue (deployment) en Kubernetes (MicroK8s) que incluye PostgreSQL como base de datos y Keycloak como servidor de identidad.

El backend expone una API REST para gestionar las operaciones principales de una tienda online. El código se organiza en cuatro módulos independientes (catalog, customers, orders, reports), cada uno con su propio schema de base de datos PostgreSQL y sus propias migraciones gestionadas con Flyway. No existen dependencias directas a nivel de base de datos entre módulos: las referencias se hacen únicamente por identificadores (UUID), y la consistencia entre módulos se garantiza en la capa de aplicación. El frontend se ejecuta en modo SPA (Single Page Application): la página se carga una sola vez en el navegador y después solo se actualizan las partes que cambian, sin recargar la página entera. La autenticación y autorización se delegan en Keycloak, un servidor de identidad externo. Keycloak se despliega en Kubernetes, compartiendo la misma instancia de PostgreSQL que el backend. El despliegue se realiza en MicroK8s, demostrando de forma básica los principios cloud-native. El frontend y el backend se exponen al exterior a través de un Ingress (nginx) bajo el dominio maryshopping.com, que enruta las peticiones API (/maryshopping) al backend y el resto (/) al frontend.

El proyecto demuestra que es posible aplicar principios de arquitectura de software y de infraestructura cloud reconocidos por la industria a escala académica, produciendo un sistema más limpio, más mantenible y más fácil de evolucionar.

# ÍNDICE

RESUMEN....2
ÍNDICE DE FIGURAS....4
Proyecto _MARYSHOPPING....5
1. Introducción....5
1.1. Descripción del proyecto y estado del arte....5
1.2. Objetivos....8
2. Análisis, diseño y desarrollo....10
2.1. Estimación de costes. Tiempo invertido. Justificación....10
2.2. Tecnologías y herramientas utilizadas (nombrarlas y pequeña descripción, justificación de selección)....15
2.3. Diseño de interfaz (diseño general de pantallas, estructura, paleta de colores, facilidades de uso, elementos destacados, etc.)....21
2.4. Concepción de la base de datos....32
2.5. Diseño y arquitectura de la solución....40
2.6. Principales dificultades encontradas y cómo se han afrontado....60
2.7. Despliegue (dónde está alojada, cómo acceder (usuarios, contraseñas), qué se necesita, etc.)....64
3. Conclusiones y recomendaciones....72
4. Bibliografía y webgrafía....73

# ÍNDICE DE FIGURAS

Figura 1: Página de login.....28
Figura 2: Tabla de informes. (Mejores clientes).....28
Figura 3: Panel del administrador.....29
Figura 4: Página catálogo de productos.....29
Figura 5: Página carrito de compras.....30
Figura 6: Notificación de pedido confirmado.....30
Figura 7: Página de perfil de usuario.....31
Figura 8: Página de pedidos del usuario registrado.....31
Figura 9: Diagrama de clases de catálogo.....36
Figura 10: Diagrama de clases orders.....37
Figura 11: Diagrama de clases customers.....37
Figura 12: Diagrama de clases reports.....38
Figura 13: Arquitectura por capas vs modular.....40
Figura 14: Organización por casos de uso.....41
Figura 15: Arquitectura limpia.....42
Figura 16: DDD (Domain-Driven-Design).....43
Figura 17: CQRS.....44
Figura 18: Mapa de acceso a la aplicación.....71

# PROYECTO _MARYSHOPPING

## 1. Introducción

### 1.1. Descripción del proyecto y estado del arte

MarySHopping es una aplicación de comercio electrónico desarrollada como Proyecto de Fin de Grado del ciclo formativo DAW (Desarrollo de Aplicaciones Web) en el C.I.F.P. Ponferrada, Castilla y León.

El proyecto tiene un doble objetivo:

Por un lado, aplicar arquitecturas y patrones de diseño de software utilizados en la industria (DDD, Vertical Slice Architecture, Monolito Modular) y por otro lado, practicar con las tecnologías y principios del ecosistema cloud-native: contenedores, orquestación con Kubernetes, programación declarativa, aplicaciones stateless, virtualización, automatización y modularización de la infraestructura.

El proyecto se organiza como un monorepo con tres carpetas:

- backend/: servicio REST en Java 25 + Spring Boot 4.1.0.
- frontend/: aplicación SPA en Nuxt 4 (Vue 3) + Vuetify 4 + TypeScript.
- deployment/: Dentro del cual tenemos: development/: entorno de desarrollo con Docker Compose y production/: entorno de producción con MicroK8s.

El backend consiste en un servicio web REST que expone una API HTTP para gestionar los datos y la lógica de negocio de una tienda online. Las funcionalidades principales que cubre son:

- Gestión del catálogo: categorías de productos, proveedores y productos con imágenes. El gestor del negocio puede crear, modificar y eliminar el catálogo.
- Gestión de clientes: registro y consulta de clientes, incluyendo sus direcciones de facturación y envío.

- Gestión de pedidos: creación de pedidos con sus líneas (productos, cantidades, precios) y consulta del histórico de compras.
- Informes y reporting: consultas agregadas sobre los datos de ventas, como los productos más vendidos o los clientes con mayor volumen de compra.

El frontend se desarrolla con Nuxt 4 (Vue 3), Vuetify 4, Pinia y TypeScript. El backend se prueba adicionalmente con Postman. Todos los componentes (backend, frontend, PostgreSQL y Keycloak) se despliegan en Kubernetes (MicroK8s), definidos como manifiestos YAML declarativos.

El frontend del proyecto está desarrollado con las siguientes tecnologías:

- Nuxt 4.4.2: framework basado en Vue 3, con soporte para SPA y SSR.
- Vue 3.5: framework JavaScript progresivo y reactivo, ampliamente utilizado.
- Vuetify 4: biblioteca de componentes UI siguiendo Material Design.
- Pinia: gestión de estado reactiva .
- TypeScript: tipado estático para mayor seguridad y autocompletado.
- Axios: cliente HTTP para consumir la API REST del backend.
- @nuxtjs/i18n: internacionalización con soporte para inglés y español.
- pnpm: gestor de paquetes rápido y eficiente en disco.

Funcionalidades del frontend:

- Catálogo de productos visible para todos los usuarios.
- Registro, login y gestión de perfil de usuario.
- Carrito de compra con persistencia (pinia-plugin-persistedstate).
- Gestión de pedidos para usuarios autenticados.
- Panel de administración (CRUD de categorías, proveedores, productos, clientes y pedidos) para el gestor del negocio.
- Integración con autenticación externa.

El frontend se ejecuta en modo SPA (Single Page Application): la página se carga una sola vez en el navegador y después solo se actualizan las partes que cambian, sin recargar la página entera. Se usa este modo por compatibilidad con Vuetify 4. Cada petición del navegador es enviada al ingress (del cluster de microk8s) ,el cual decide a qué servicio enviar dicha petición según la ruta.

El Deployment se realiza en Kubernetes (MicroK8s) con manifiestos declarativos(.yaml), aplicando de forma básica los principios cloud-native (stateless, virtualización y contenedores, replicación, automatización, programación declarativa y modularización).

**A quién va dirigida**: El gestor del negocio (administrador) que necesita gestionar el catálogo, los clientes y consultar informes de ventas. Los clientes de la tienda que, a través del frontend, pueden ver el catálogo, registrarse, gestionar su carrito y realizar pedidos.

**En cuanto al estado del arte**:

Las aplicaciones de comercio electrónico son uno de los dominios más comunes en el desarrollo web. Existen aplicaciones como WooCommerce (para WordPress), que permiten montar una tienda online sin necesidad de programar. Sin embargo, son soluciones genéricas que no siempre se adaptan a necesidades específicas.

En el ámbito del desarrollo de aplicaciones web a medida, el enfoque habitual hasta hace unos años era el modelo “Database-First”: diseñar primero las tablas de la base de datos y construir el código alrededor de ese esquema. Esto es comprensible cuando el sistema es pequeño y estable, pero tiene limitaciones importantes cuando el sistema crece o cuando los requisitos cambian con frecuencia.

En los últimos años, la industria del software ha adoptado de forma creciente metodologías como Domain-Driven Design (DDD) y patrones como Vertical Slice Architecture, que proponen organizar el código alrededor de los casos de uso y las reglas del negocio, dejando la base de datos como un detalle de implementación.

Uno de los retos más complejos de estas arquitecturas es conseguir un aislamiento real entre los bounded contexts (que cada módulo sea verdaderamente independiente, sin dependencias ocultas). En la práctica, esto implica decisiones como renunciar a las claves foráneas entre módulos, duplicar datos mediante snapshots y diseñar interfaces de comunicación

(internal_api) que no filtren detalles internos, decisiones que afronta este proyecto.

En el contexto actual de la inteligencia artificial, el desarrollo de software está cambiando. Herramientas de IA pueden generar código funcional a partir de instrucciones. Sin embargo, lo que la IA no puede decidir es la arquitectura: cómo dividir un sistema, qué datos pertenecen a qué módulo, cómo gestionar la comunicación entre módulos o cómo diseñar un sistema que sea mantenible a largo plazo.En cuanto a este tema, lo que aporta valor a un desarrollador es su capacidad para diseñar la estructura del sistema, que es lo que este proyecto quiere demostrar.

El resultado es un proyecto que, aunque académico, aplica patrones y prácticas de arquitectura de software e infraestructura cloud que se utilizan en proyectos reales de software empresarial.

## 1.2. Objetivos.

**OBJETIVO GENERAL**

El objetivo principal de este proyecto es desarrollar una aplicación de comercio electrónico completa (backend, frontend e infraestructura de despliegue) aplicando arquitecturas y patrones de diseño de software ampliamente adoptados en la industria, principios cloud-native y documentar las decisiones tomadas de forma que sirva como referencia de aprendizaje y como demostración de las competencias adquiridas durante el ciclo formativo DAW.

**OBJETIVOS ESPECÍFICOS**

Implementar una API REST funcional con Spring Boot que cubra las operaciones necesarias para gestionar una tienda online:

- CRUD completo del catálogo (categorías, proveedores, productos, imágenes).
- CRUD completo de clientes con gestión de direcciones (facturación y envío).
- CRUD completo de pedidos y líneas de pedido.
- Consultas de reporting (productos más vendidos, clientes con mayor gasto).

Aplicar Domain-Driven Design (DDD) en los módulos principales:

- Definir entidades de dominio con value objects (Address, Email, etc.).
- Separar la capa de dominio de la capa de infraestructura (persistencia).
- Usar Vertical Slice Architecture para organizar el código por caso de uso

Organizar el sistema en módulos (bounded contexts) independientes:

- catalog, customers, orders, reports, shared_kernel.
- Cada módulo con su propio esquema de base de datos PostgreSQL.
- Sin claves foráneas entre módulos (referencias lógicas por UUID).

Gestionar las migraciones de base de datos de forma controlada:

- Usar Flyway con configuración multi-módulo personalizada.
- Una instancia de Flyway por módulo, apuntando a su propio schema.

Incluir un módulo de reporting diferenciado (reports):

- Sin capa de dominio, siguiendo el principio CQRS para lectura.
- Tabla denormalizada como read model para consultas agregadas eficientes.

Documentar el proyecto de forma académica:

- Justificar las decisiones de diseño con fuentes reconocidas.
- Generar diagramas (flujo, casos de uso, secuencia, clases, MER).
- Defender el enfoque adoptado frente al enfoque “Database-First”.

Preparar el sistema para despliegue con Docker:

- Incluir Dockerfile para contenerizar la aplicación.

Diseñar e implementar un sistema de seguridad RBAC con JWT:

- Definir los roles del sistema (ROLE_ANONYMOUS, ROLE_CUSTOMER, ROLE_ADMIN).
- Especificar los permisos de cada rol sobre cada endpoint de la API.
- Implementar autenticación stateless con JWT mediante Spring Security.
- Aplicar restricciones de propiedad (un cliente solo accede a sus propios datos).

Desarrollar el frontend de la aplicación con Nuxt 4 (Vue 3):

- Implementar la interfaz de usuario con Vuetify 4 (Material Design).
- Catálogo de productos, carrito de compra, gestión de pedidos y perfil de usuario.
- Panel de administración completo (CRUD de categorías, proveedores, productos, clientes y pedidos).
- Gestión de estado con Pinia (carrito persistente entre sesiones).
- Tipado estático con TypeScript para mayor seguridad.

Desplegar toda la infraestructura en Kubernetes (MicroK8s):

- Aplicar principios cloud-native de forma básica.
- Desplegar PostgreSQL con volumen persistente (PVC).
- Desplegar Keycloak mediante el Operator (patrón Operator de Kubernetes con Custom Resource Definitions).
- Desplegar el backend y frontend en kubernetes.

**OBJETIVOS DE APRENDIZAJE PERSONAL**

- Comprender y aplicar los principios de DDD en un proyecto real.
- Entender qué es un bounded context y cómo separar módulos de forma efectiva, afrontando la complejidad del aislamiento real entre contextos.
- Aprender a usar Spring Boot, Spring Data JPA, Flyway y Lombok en combinación.
- Aprovechar las funcionalidades de Java 25 que se usan en el proyecto (records y streams) para implementar los patrones arquitectónicos con menos código repetitivo y de forma más clara.
- Ganar experiencia con herramientas profesionales (Java 25, Maven, Docker, Postman, Kubernetes, MicroK8s).
- Entender los principios básicos de la infraestructura cloud-native: qué significa stateless, por qué se usan contenedores, cómo funciona la orquestación, y por qué se define la infraestructura de forma declarativa.
- Entender el valor de la arquitectura de software en un contexto donde la generación de código se automatiza cada vez más.

# 2. Análisis, diseño y desarrollo

## 2.1. Estimación de costes. Tiempo invertido. Justificación.

**ESTIMACIÓN DE TIEMPO INVERTIDO**

El desarrollo del proyecto se ha dividido en las siguientes fases, con una estimación aproximada del tiempo dedicado a cada una:

FASE 1 — Diseño y planificación de la arquitectura

- Estudio de DDD, Vertical Slice Architecture y patrones de módulos.
- Definición de los bounded contexts y sus responsabilidades.
- Diseño del esquema de base de datos por módulo.
- Diseño de la estructura de carpetas y convenciones de nomenclatura. Tiempo estimado: 15-20 horas.

FASE 2 — Configuración inicial del proyecto

- Inicialización del proyecto Spring Boot con Maven.
- Configuración de dependencias (Spring Data JPA, Flyway, Lombok, PostgreSQL).
- Configuración multi-módulo de Flyway (FlywayConfig.java, MultiModuleFlywayMigrationStrategyConfig.java).
- Configuración de application.yaml.
- Creación del Dockerfile. Tiempo estimado: 8-12 horas.

FASE 3 — Implementación del módulo catalog

- Definición del dominio (Category, Provider, Product y sus value objects).
- Implementación de todos los casos de uso (CRUD de categorías, proveedores, productos e imágenes) con vertical slice completo.
- Migración Flyway del schema catalog. Tiempo estimado: 20-25 horas.

FASE 4 — Implementación del módulo customers

- Definición del shared_kernel (Address y value objects).
- Implementación de todos los casos de uso (CRUD de clientes).
- Migración Flyway del schema customers. Tiempo estimado: 12-15 horas

FASE 5 — Implementación del módulo orders

- Definición del dominio (Order, OrderLine y sus value objects).
- Implementación de todos los casos de uso (CRUD de pedidos y líneas de pedido).
- Migración Flyway del schema orders. Tiempo estimado: 20-25 horas

FASE 6 — Implementación del módulo reports

- Diseño de la tabla denormalizada order_report.
- Implementación de casos de uso: get_top_products, get_top_customers, save_order_confirmed (recepción de eventos desde el módulo orders).
- Migración Flyway del schema reports. Tiempo estimado: 10-15 horas

FASE 7 — Pruebas con Postman

- Creación de colecciones de Postman para cada endpoint.
- Verificación de todos los casos de uso. Tiempo estimado: 8-10 horas

FASE 8 — Documentación

- Generación de diagramas (flujo, casos de uso, secuencia, clases, MER).
- Redacción de la defensa de la arquitectura.
- Redacción de todos los apartados del trabajo final. Tiempo estimado: 15-20 horas

FASE 9 — Desarrollo del frontend (Nuxt 4 / Vue 3)

- Inicialización del proyecto Nuxt 4 con pnpm.
- Configuración de Vuetify 4 (Material Design), Pinia (estado) y TypeScript.
- Implementación de las páginas principales: catálogo de productos, detalle de producto, carrito de compra, pedidos, perfil de usuario, registro y login.
- Implementación del panel de administración: CRUD de categorías, proveedores, productos, clientes y pedidos.
- Servicios de API (Axios): catalogProductsService, categoriesService, customersService, ordersService, providersService, authService.
- Stores (Pinia): cartStore (con persistencia), userStore.
- Modelos TypeScript para productos, pedidos, clientes. Tiempo estimado: 30-40 horas.

FASE 10 — Infraestructura de despliegue (Kubernetes / MicroK8s)

- Instalación y configuración de MicroK8s.
- Diseño de la estructura de namespaces (postgres, keycloak, apps).
- Manifiestos de PostgreSQL: ConfigMap, PVC, Deployment + Service (NodePort).

- Manifiestos de Keycloak: CRDs del operador, instalación del Keycloak Operator, DB Secret, instancia Keycloak via Custom Resource.
- Manifiestos del backend: ConfigMap (application.yaml externalizado), Deployment + Service (NodePort).
- Manifiestos del frontend: ingress, Deployment + Service + ConfigMap.
- Script de creación de secret Docker Registry para GHCR.
- Aprendizaje de conceptos: Pods, Deployments, Services, ConfigMaps, Secrets, PVCs, Namespaces, Operators, CRDs. Tiempo estimado: 20-30 horas.

---

TIEMPO TOTAL ESTIMADO: 158-212 horas

(backend: 108-142 h + frontend: 30-40 h + infraestructura: 20-30 h)

**ESTIMACIÓN DE COSTES**

Este proyecto tiene un coste económico prácticamente nulo, ya que todas las herramientas utilizadas son de software libre o tienen licencia gratuita:

COSTE DEL SOFTWARE:

- Java 25 (OpenJDK): gratuito y de código abierto.
- Spring Boot 4.1.0: gratuito y de código abierto.
- PostgreSQL: gratuito y de código abierto.
- Flyway Community Edition: gratuito para proyectos no comerciales.
- Lombok: gratuito y de código abierto.
- Maven: gratuito y de código abierto.
- Node.js 22 LTS / pnpm 10.x: gratuito y de código abierto.
- Nuxt 4 / Vue 3: gratuito y de código abierto.
- Vuetify 4: gratuito y de código abierto.
- TypeScript: gratuito y de código abierto.
- Docker (Docker Desktop): gratuito para uso personal/educativo.

- Microk8s(versión ligera de kubernetes): gratuita.
- IntelliJ IDEA Community Edition o versión de estudiante: gratuita.
- VSC(visual studio code): gratuita.
- Postman: gratuito para uso individual.
- Git / GitHub: gratuito para repositorios públicos.

COSTE DE HARDWARE:

- El proyecto se desarrolla en el ordenador personal del estudiante. No se requiere hardware adicional.

---

COSTE TOTAL DEL DESARROLLO : 0 € (herramientas 100% gratuitas).

**JUSTIFICACIÓN DEL TIEMPO Y DEL COSTE**

El tiempo invertido puede parecer elevado para un proyecto de fin de grado, pero se justifica por:

- La curva de aprendizaje de DDD y Vertical Slice Architecture: son conceptos nuevos que requieren tiempo para entender e implementar correctamente.
- La configuración multi-módulo de Flyway: no es una configuración estándar y requirió investigación y desarrollo específico.
- La calidad del código: se ha priorizado escribir código limpio, bien organizado y consistente, lo que implica más tiempo de diseño y revisión.
- La infraestructura de despliegue: aprender Kubernetes (conceptos, manifiestos, el Keycloak Operator, PVCs, namespaces) y configurar MicroK8s requiere un tiempo considerable de aprendizaje para quien no ha trabajado antes con orquestación de contenedores.
- La documentación académica: justificar las decisiones con fuentes reconocidas y generar diagramas completos requiere tiempo adicional.

La elección de herramientas gratuitas y de código abierto no solo reduce el coste a cero, sino que también está alineada con el perfil de un desarrollador

web que trabaja con las tecnologías más utilizadas en el mercado laboral actual.

## 2.2. Tecnologías y herramientas utilizadas (nombrarlas y pequeña descripción, justificación de selección)

### 1. LENGUAJE: JAVA 25

Java es uno de los lenguajes de programación más utilizados en el desarrollo de aplicaciones de servidor (backends). En su versión 25, Java incluye varias funcionalidades que se usan directamente en este proyecto:

- Records: son un tipo especial de clase que es inmutable (una vez creada, no se puede cambiar). Java genera automáticamente el constructor, los getters, equals(), hashCode() y toString(). En la práctica, esto significa que en lugar de escribir una clase de 40-50 líneas para un objeto con 5 campos, un record lo hace en una sola línea.
- Streams API: permite procesar listas de datos de forma declarativa. En lugar de escribir un bucle for para recorrer una lista y filtrar elementos.
- Programación funcional: lambdas (funciones sin nombre que se escriben directamente donde se necesitan) y method references (una forma abreviada de referirse a un método existente). Estas herramientas permiten escribir código más corto y más fácil de leer.

**Justificación**: Java es el lenguaje del ecosistema Spring Boot. Según el informe “Developer Survey” de Stack Overflow (2024), Java se mantiene entre los 5 lenguajes más utilizados profesionalmente. El informe de JetBrains (2024) lo sitúa como el segundo lenguaje más usado para desarrollo backend. En España, según portales como InfoJobs y LinkedIn (consulta febrero 2026), Java/Spring Boot es la combinación más demandada en ofertas para desarrolladores backend.

### 2. FRAMEWORK: SPRING BOOT 4.1.0

Spring Boot es el framework más utilizado para desarrollar aplicaciones web con Java. Simplifica enormemente la configuración: con muy poco código de configuración se tiene un servidor web funcionando.

Qué hace Spring Boot por nosotros :

- Gestión de conexiones a la base de datos: Spring mantiene una reserva de conexiones y las abre, reutiliza y cierra automáticamente.
- Transacciones automáticas: si algo falla al guardar en la base de datos, Spring deshace los cambios sin que tengamos que escribir código para ello.
- Conversión automática de JSON: convierte los objetos Java a JSON (para enviarlos al frontend) y viceversa, sin que escribamos ningún código de serialización.
- Mapeo de rutas HTTP: asocia las URLs (como /customers/{id}) al método correcto del controller, gracias a anotaciones como @GetMapping, @PostMapping, etc..
- Inyección de dependencias: crea los objetos (services, repositories) y los conecta entre sí automáticamente.
- Autoconfiguración: Spring Boot detecta las dependencias del proyecto y configura automáticamente lo necesario.

**Justificación**: Spring Boot es utilizado por más del 60% de los desarrolladores Java profesionales según JetBrains (2024). Es el framework más solicitado en ofertas de trabajo Java.

### 3. BASE DE DATOS: POSTGRESQL

PostgreSQL es un sistema de base de datos relacional gratuito y de código abierto. Se caracteriza por:

- Soporte de tipos avanzados (UUID, JSON, arrays, etc.).
- Soporte de schemas: espacios de nombres dentro de la misma base de datos, lo que permite que cada módulo del proyecto tenga su propio “espacio” aislado.
- Rendimiento sólido en consultas complejas.
- Completamente gratuito.

**Justificación**: Es ideal para mi proyecto porque soporta Schemas independientes: Cada módulo (customers, catalog, orders) tiene su propio "espacio" en la misma base de datos, lo que simula que cada uno tiene su propia BD (perfecto para mi arquitectura modular). Soporta tipos avanzados y Spring JPA trabaja muy bien con ellos. Gratuito y robusto. Compatible con

Spring Boot y Flyway: Funciona perfectamente con las herramientas que ya uso (Spring Data JPA y Flyway para migraciones).

## 4. ORM Y PERSISTENCIA: SPRING DATA JPA (HIBERNATE)

Spring Data JPA es la herramienta que permite trabajar con la base de datos de forma orientada a objetos, sin tener que escribir SQL a mano en la mayoría de los casos.

JPA (Java Persistence API) es una especificación de Java que define cómo mapear objetos Java a tablas de la base de datos. Cada campo de la clase Java corresponde a una columna de la tabla. Las anotaciones @Entity, @Table, @Id, @Column definen esta correspondencia.

Hibernate es la implementación más utilizada de JPA. Es el “motor” que traduce las operaciones de JPA a consultas SQL reales contra la base de datos. Spring Boot lo incluye por defecto.

**Justificación**: Spring Data JPA es el estándar en Java para no escribir SQL a mano. Mapea objetos Java a tablas automáticamente, genera consultas y se integra perfecto con Spring Boot.

## 5. MIGRACIONES: FLYWAY

Flyway es una herramienta para gestionar los cambios en la estructura de la base de datos de forma controlada y versionada. Una migración es un script SQL que describe un cambio en la estructura de la base de datos. En lugar de modificar la base de datos manualmente, se escriben estos scripts y Flyway los aplica automáticamente al arrancar la aplicación.

**Justificación**: La posibilidad de errores es mucho menor que si se hiciera de manera manual. Permite gestionar migraciones separadas para cada módulo (puesto que mi arquitectura es multi-módulo con 4 schemas independientes) sin conflictos.

## 6. REDUCCIÓN DE CÓDIGO REPETITIVO: LOMBOK

Lombok es una biblioteca que genera automáticamente código Java repetitivo, necesario, pero que siempre es igual y no aporta valor, como getters, setters, constructores, equals/hashCode. En este proyecto se usa para: @Builder: genera el patrón Builder para construir objetos paso a paso, @AllArgsConstructor, @NoArgsConstructor: genera constructores y @Getter: genera métodos getter.

**Justificación**: reduce la cantidad de código repetitivo en las entidades JPA, permitiendo escribir menos líneas sin perder legibilidad.

## 7. GESTOR DE DEPENDENCIAS: MAVEN

Maven es la herramienta que gestiona las dependencias (las librerías externas que necesita el proyecto) limpia, compila el código y genera el fichero .jar ejecutable.

Todas las dependencias se declaran en un fichero XML llamado pom.xml. Maven las descarga automáticamente de un repositorio central (Maven Central) junto con todas las librerías que esas dependencias necesitan a su vez (dependencias transitivas).

**Justificación**: automatiza la descarga de dependencias y la construcción del aplicativo (.jar), evitando tener que configurar manualmente cada librería.

## 8. CONTENERIZACIÓN: DOCKER

Docker permite empaquetar la aplicación y todo lo que necesita en un “contenedor” que se puede ejecutar en cualquier máquina sin tener que instalar Java ni configurar nada.

**Justificación**: portabilidad(sin importar el sistema operativo), aislamiento(cada componente corre en su propio contenedor), facilita el despliegue(sin configuración manual) y reproducibilidad(todos los desarrolladores usan la misma imagen). Sin Docker tendríamos que instalar y configurar todo manualmente en cada máquina.

## 9. PRUEBAS MANUALES: POSTMAN

Postman es una herramienta que permite enviar peticiones HTTP a una API REST y ver las respuestas. Se ha usado para probar todos los endpoints del proyecto.

Se incluye en el repositorio el fichero postman_create_customer.json con ejemplos.

**Justificación**: sin Postman, tendría que comprobar los endpoints usando una terminal, lo cual es mucho mas lento e incómodo.

## 10. FRONTEND: NUXT 4 (VUE 3) + VUETIFY 4

Mi frontend se ejecuta en modo SPA (Single Page Application). Una SPA es una aplicación web que se carga una sola vez en el navegador. A partir de ese

momento, cuando el usuario navega por las páginas, el navegador no recarga la página entera: JavaScript actualiza solo las partes que cambian. Esto hace que la navegación sea más rápida y fluida. Nuxt 4 soporta (SPA). Las tecnologías del frontend son:

- Nuxt 4.4.2: framework basado en Vue 3, con soporte para SPA.
- Vue 3.5: framework JavaScript progresivo y reactivo.
- Vuetify 4: biblioteca de componentes UI siguiendo Material Design.
- Pinia: gestión de estado reactiva.
- TypeScript: tipado estático para mayor seguridad.
- Axios: cliente HTTP para consumir la API REST del backend.
- @nuxtjs/i18n: internacionalización con soporte para inglés y español.
- pnpm: gestor de paquetes.

**Justificación**: Vue.js es un framework JavaScript progresivo, ampliamente utilizado y bien documentado, con una curva de aprendizaje accesible. Según la encuesta “State of JavaScript” (2024) y el “Developer Survey” de Stack Overflow (2024), Vue.js se mantiene entre los 3 frameworks frontend más utilizados y con mayor satisfacción por parte de los desarrolladores. Vuetify proporciona componentes profesionales listos para usar, reduciendo el tiempo de desarrollo del frontend. Pinia es el gestor de estado oficial recomendado por el equipo de Vue.js. TypeScript añade seguridad de tipos a JavaScript, reduciendo errores en tiempo de ejecución. Usamos el modo SPA porque la navegación es mas rápida una vez cargada la página, transacciones suaves sin recarga visible y porque Vuetify 4 lo requiere por compatibilidad.

## 11. GESTIÓN DE IDENTIDAD: KEYCLOAK

Keycloak es un servidor de gestión de identidad y acceso (IAM — Identity and Access Management) de código abierto. Implementa los estándares OAuth2 y OpenID Connect. Keycloak :

- Almacena usuarios, credenciales y roles en su propia base de datos.
- Emite tokens JWT firmados con RSA cuando un usuario se autentica.
- Proporciona una API de administración para gestionar usuarios desde el backend (crear, modificar, eliminar, asignar roles).

- Desacopla la autenticación del código del backend: Spring Security solo necesita validar los tokens JWT emitidos por Keycloak.

En este proyecto, Keycloak comparte la instancia de PostgreSQL con el backend (pero usa sus propias tablas en un schema independiente).

**Justificación**: Keycloak es la solución IAM de código abierto más utilizada en el ecosistema Java/Spring. Externalizar la gestión de identidad es una buena práctica reconocida por la industria.

## 12. ORQUESTACIÓN: KUBERNETES (MICROK8S) Y PRINCIPIOS CLOUD-NATIVE

Kubernetes (K8s) es la plataforma estándar de la industria para orquestar contenedores en producción. MicroK8s es una distribución ligera de Kubernetes, ideal para desarrollo local y proyectos académicos.

Los principios cloud-native son un conjunto de buenas prácticas para diseñar y desplegar aplicaciones en entornos de contenedores. Este proyecto los aplica de forma básica y didáctica:

- STATELESS (sin estado): La aplicación backend no guarda estado en memoria entre peticiones HTTP. Toda la información se persiste en PostgreSQL. Esto significa que si el contenedor del backend se destruye y Kubernetes crea uno nuevo, no se pierde ningún dato. Es el requisito fundamental para poder escalar horizontalmente (tener varias copias del backend).

- VIRTUALIZACIÓN Y CONTENEDORES: Cada componente se empaqueta en un contenedor Docker aislado, con sus propias dependencias. Un contenedor tiene su propio sistema de ficheros y sus propias librerías, pero comparte el kernel del sistema operativo con el servidor (por eso es más ligero que una máquina virtual completa).

- REPLICACIÓN: Kubernetes permite aumentar el número de réplicas de un Deployment con un solo comando.

- AUTOMATIZACIÓN: Si un contenedor falla, Kubernetes lo detecta y lo reinicia automáticamente. También permite actualizaciones sin corte de servicio: va creando contenedores nuevos con la nueva versión y eliminando los antiguos uno a uno, sin que el usuario note interrupción.

- PROGRAMACIÓN DECLARATIVA: Toda la infraestructura se define en ficheros YAML. En lugar de ejecutar comandos para instalar y configurar

cada componente, se describe el estado deseado ("quiero 1 réplica del backend con esta imagen Docker, conectada a esta base de datos") y Kubernetes se encarga de hacerlo realidad. Si algo se desvía del estado deseado, Kubernetes lo corrige automáticamente.

- MODULARIZACIÓN: Los componentes se aíslan en namespaces diferentes (postgres, keycloak, apps). Cada namespace es como una "carpeta" que contiene sus propios recursos, aislados de los demás. Esto refleja en la infraestructura la misma separación de responsabilidades que se aplica en el código (bounded contexts).

## 2.3. Diseño de interfaz (diseño general de pantallas, estructura, paleta de colores, facilidades de uso, elementos destacados, etc.).

**PRINCIPIOS DE DISEÑO**

El diseño del frontend sigue principios de interfaz moderna y responsive:

- Material Design 3: Vuetify 4 implementa Material Design.El desarrollador no necesita diseñar componentes desde cero: Vuetify proporciona botones, tablas, formularios, tarjetas, diálogos, etc., ya estilizados y con comportamiento responsive integrado.
- Responsive Design: La interfaz se adapta automáticamente a dispositivos de diferentes tamaños (móviles, tablets,) sin perder funcionalidad ni legibilidad, gracias al sistema de grid de Vuetify.
- Usabilidad intuitiva: Los controles están organizados de forma lógica y visible. La navegación es consistente en toda la aplicación.
- Accesibilidad: Se siguen buenas prácticas de accesibilidad web (contraste de colores, labels en formularios, navegación por teclado).

**ESTRUCTURA GENERAL DE LA APLICACIÓN**

La aplicación es una SPA (Single Page Application): se carga una sola vez en el navegador y la navegación entre pantallas se realiza sin recargar la página, actualizando solo las partes que cambian.

La interfaz se organiza en dos layouts principales:

a) **Layout Público (sin autenticación)**:

- Encabezado (navbar) con logo de MarryShopping y enlaces de navegación (inicio, catálogo).

- Área de contenido principal.

- Botones de login/registro visibles en la barra de navegación.

b) **Layout Autenticado (con token JWT):**

- Encabezado mejorado con menú de usuario, acceso al carrito de compra y perfil.

- Barra lateral de navegación con opciones según el rol del usuario:

  Rol "customer": Catálogo, Mi carrito, Mis pedidos, Mi perfil, Cerrar sesión.

  Rol "administrator": Panel de control, Gestión de catálogo (categorías, productos, proveedores), Gestión de clientes, Gestión de pedidos, Reportes y Cerrar sesión.

- Área de contenido principal.

**COMPONENTES VUETIFY UTILIZADOS**

v-app-bar ----------Barra de navegación superior (navbar)

v-navigation-drawer ----------Barra lateral de navegación

v-card ----------Paneles informativos (producto, pedido, cliente)

v-btn ----------Botones de acción (enviar, cancelar, guardar, eliminar)

v-text-field----------Campos de entrada de texto (email, contraseña, búsqueda)

v-select ----------Listas desplegables (categorías, provincias)

v-data-table ----Tablas de datos con paginación (listados de productos, pedidos, clientes)

v-dialog ----------Ventanas modales (confirmaciones, formularios emergentes)

v-snackbar----------Notificaciones breves en esquina de pantalla (éxito, error)

v-progress-circular ----------Indicadores de carga (spinners)

v-chip ---------Etiquetas pequeñas de estado (estado del pedido, rol del usuario)

v-icon ----------Iconos de acción (editar, eliminar, ver detalle, carrito)

v-stepper ---------- ----------Pasos secuenciales (flujo de compra)

v-carousel ----------Carrusel de imágenes (productos destacados en home)

v-form ----------Formularios con validación integrada

v-footer ----------Pie de página

**PANTALLAS POR ROL DE USUARIO**

1. **USUARIO NO AUTENTICADO**

   El usuario sin sesión puede acceder a:

   - Home: Página de bienvenida con carrusel de productos destacados y accesos rápidos al catálogo.

   - Catálogo: Ver todas las categorías de productos, explorar productos dentro de cada categoría, ver detalles de un producto (descripción, precio, proveedor, imágenes). No puede añadir productos al carrito (se le solicita iniciar sesión).

   - Login: Formulario con dos campos (email y contraseña) y botón "Iniciar sesión".

   - Registro: Formulario completo con nombre, apellido, DNI, teléfono, email, contraseña, dirección de facturación y dirección de envío.

2. **USUARIO CON ROL "CUSTOMER"**

   El cliente autenticado accede a todo lo anterior más:

   - Catálogo con carrito: Puede añadir productos al carrito. El icono del carrito en la navbar muestra el número de artículos.

   - Carrito de compra: Listado de productos añadidos (nombre, precio, cantidad), modificar cantidades, eliminar productos, cálculo automático del total y botónes "Confirmar compra" y "Vaciar carrito".

   - Mis pedidos: Listado de pedidos del cliente con ID, fecha, estado y total.

- Detalle de pedido: ID, fecha, estado, dirección de envío, líneas del pedido (producto, cantidad, precio unitario, subtotal) y total calculado.

- Mi perfil: Datos personales y direcciones, con opción de editar y botones de “Eliminar cuenta” y “Guardar cambios”.

- Cerrar sesión: Revoca el token JWT en el servidor y limpia el estado local.

### 3. USUARIO CON ROL “ADMINISTRATOR”

El administrador accede a funciones adicionales:

- Panel de control: Estadísticas rápidas y gráficos de resumen (productos más vendidos, clientes top).

- Gestión de catálogo: CRUD completo de categorías, productos (con selector de categoría, proveedor, referencia del proveedor, precio, descripción y subida de imágenes) y proveedores.

- Gestión de clientes: Listar todos los clientes (tabla paginada), ver detalles, crear, editar y eliminar clientes.

- Gestión de pedidos: Listar todos los pedidos de cualquier cliente, ver detalles.

- Reportes: Top N productos más vendidos y Top N clientes por volumen de compra.

## PALETA DE COLORES

### Colores principales

El diseño de MaryShopping utiliza la paleta de colores de Vuetify 4 personalizada para la marca:

**Primario** -------#6366f1(indigo)---------Botones principales,enlaces,encabezados destacados.

**Secundario** ----------#8b5cf6(violeta)--------Elementos secundarios, badges, destacados.

**Éxito** ----------#10b981(verde)----------Confirmaciones, guardado exitoso, estado CONFIRMED.

**Error** ----------#ef4444(rojo)---Mensajes de error, botones peligrosos (eliminar).

**Advertencia** ----------#f59e0b(ámbar)-----Campos obligatorios, avisos, estado PENDING.

**Información** ----------#3b82f6(azul)----------Información adicional,mensajes informativos.

**Colores neutros(Tema oscuro por defecto)**

**Background** ----------(#0a0a0f)----------Fondo de página.

**Surface** ----------(#12121a)----------Tarjetas,modales, paneles.

**Surface light** ---------- (#1a1a24)----------Fondos secundarios.

**Surface variant**---------(#22222e)----------Filas alternas,separadores.

**Texto principal** --------(#f1f5f9)----------Texto sobre fondo(on-background).

**Texto secundario** -----(#e2e8f0)----------Texto sobre superficie(on-surface).

**Colores neutros(Tema claro)**

**Background** ----------(#f8fafc)----------Fondo de página.

**Surface** ----------(#ffffff)----------Tarjetas,modales, paneles.

**Surface light** ---------- ---(#f1f5f9)----------Fondos secundarios.

**Surface variant**---------(#e2e8f0)----------Filas alternas,separadores.

**Texto principal** --------(#0f172a)-Texto sobre fondo(on-background).

**Texto secundario** -----(#1e293b)----------Texto sobre superficie(on-surface).

**Tema claro y tema oscuro**

Vuetify 4 soporta temas claro y oscuro. La aplicación implementa ambos modos con un selector en la barra de navegación.

**FACILIDADES DE USO**

**Búsqueda y filtrado**

- Barra de búsqueda global en la navbar que filtra productos por nombre en tiempo real.

- Filtros en listados: por categoría y ordenación por nombre o precio (de menor a mayor y viceversa).

**Paginación**

- Las tablas de datos usan paginación con controles “Anterior/Siguiente” y selector de número de elementos por página, proporcionados por v-data-table de Vuetify.

**Carrito persistente**

- El carrito de compra se guarda en localStorage del navegador gracias al plugin pinia-plugin-persistedstate. Esto permite que el carrito se conserve aunque el usuario cierre el navegador.
- El número de artículos se muestra en el icono del carrito de la navbar.

**Indicadores de estado**

- Chips de color para el estado de pedidos:,naranja para CREATED y verde para CONFIRMED.
- Iconos intuitivos para las acciones: lápiz (editar), papelera (eliminar), ojo (ver detalle), carrito (añadir).

**ELEMENTOS DESTACADOS DE LA INTERFAZ**

**Tarjeta de producto (ProductCard)**

Cada producto se muestra en una tarjeta (v-card) con: imagen de portada, nombre del producto, precio destacado, descripción del producto y botón “Añadir al carrito”.

**Tabla de pedidos con acciones rápidas con rol administrador**

En “Ver pedidos”, la tabla (v-data-table) muestra: ID para ver detalle, fecha, estado como chip de color, total con dos decimales y columna de acciones con iconos.

**Formularios de login y registro**

- Login: con 2 campos (email, contraseña) y enlace a registro.
- Registro: formulario completo con validación en tiempo real, incluyendo dos secciones de dirección (facturación y envío).

**INTERNACIONALIZACIÓN (i18n)**

El frontend soporta dos idiomas: español (es) e inglés (en), gracias al módulo @nuxtjs/i18n.

- Selector de idioma en la barra de navegación.
- Todos los textos estáticos de la interfaz se cargan desde archivos JSON (es.json, en.json) en la carpeta i18n/.
- El idioma seleccionado se guarda en localStorage y se mantiene entre sesiones.

**INTEGRACIÓN CON EL BACKEND**

El frontend se conecta con la API REST del backend mediante:

- Servicio Axios: Un archivo centralizado configura la URL base del backend y añade automáticamente el token JWT en el header Authorization de cada petición HTTP.
- Stores (Pinia): Cada módulo(catalogo, pedidos, clientes) tiene un store que almacena los datos en memoria para evitar peticiones redundantes.
- Manejo de errores HTTP:

  - Error 401 (no autenticado): redirige automáticamente a la pantalla de login.

  - Error 403 (no autorizado): muestra mensaje “Sin permiso para esta acción”.

  - Error 409 (conflicto): muestra el mensaje de validación de negocio devuelto por el backend.

  - Error 500 (error del servidor): muestra mensaje genérico de error.

**RENDIMIENTO Y OPTIMIZACIONES**

- Lazy loading: Las páginas de administración se cargan bajo demanda (solo cuando el usuario navega a ellas), reduciendo el tiempo de carga inicial.
- Code splitting: Nuxt genera automáticamente un archivo JavaScript por página que se cargan cuando hacen falta, en lugar de enviar todo el código de golpe.
- Caché de datos: Los resultados de las peticiones GET se almacenan en los stores de Pinia, evitando peticiones repetidas al backend.
- Compresión de imágenes: Las imágenes de productos se sirven en formatos optimizados.

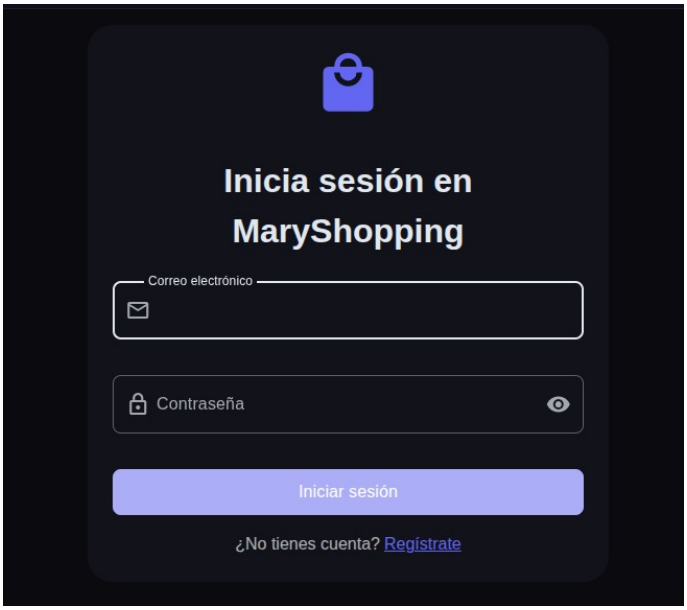
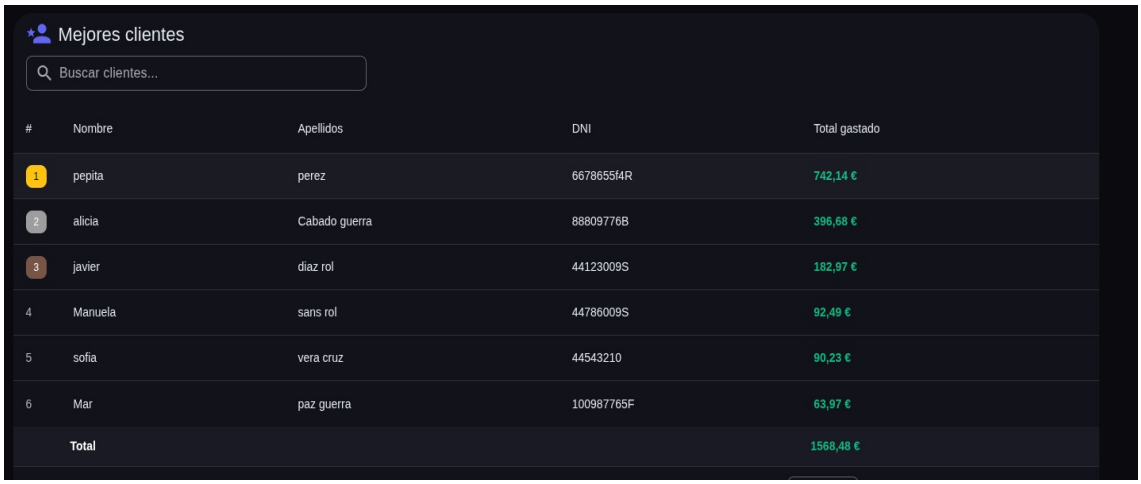



**Figura 1: Página de login.**




| # | Nombre | Apellidos | DNI | Total gastado |
|---|---|---|---|---|
| 1 | pepita | perez | 6678655f4R | 742,14 € |
| 2 | alicia | Cabado guerra | 88809776B | 396,68 € |
| 3 | javier | diaz rol | 44123009S | 182,97 € |
| 4 | Manuela | sans rol | 44786009S | 92,49 € |
| 5 | sofia | vera cruz | 44543210 | 90,23 € |
| 6 | Mar | paz guerra | 100987765F | 63,97 € |
| | Total | | | 1568,48 € |

**Figura 2: Tabla de informes. (Mejores clientes)**

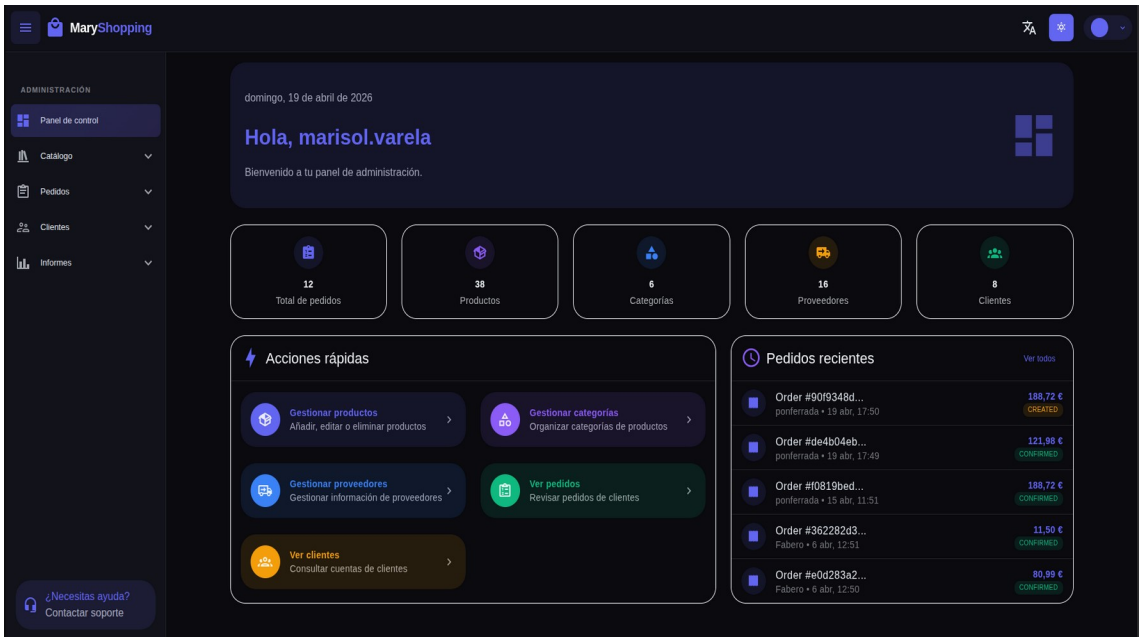



**Figura 3: Panel del administrador.**

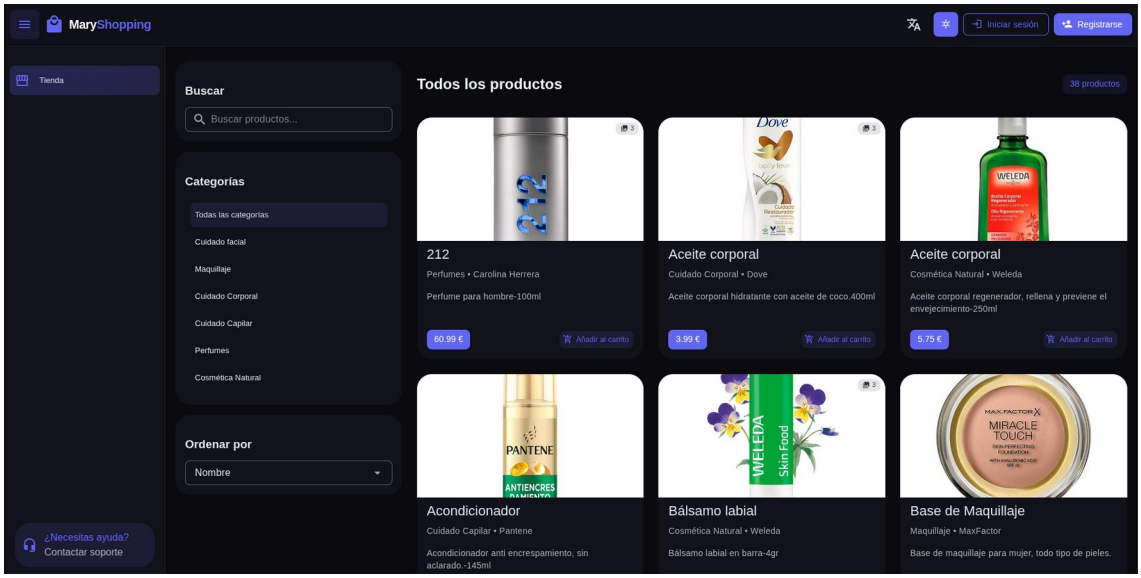



**Figura 4: Página catálogo de productos.**

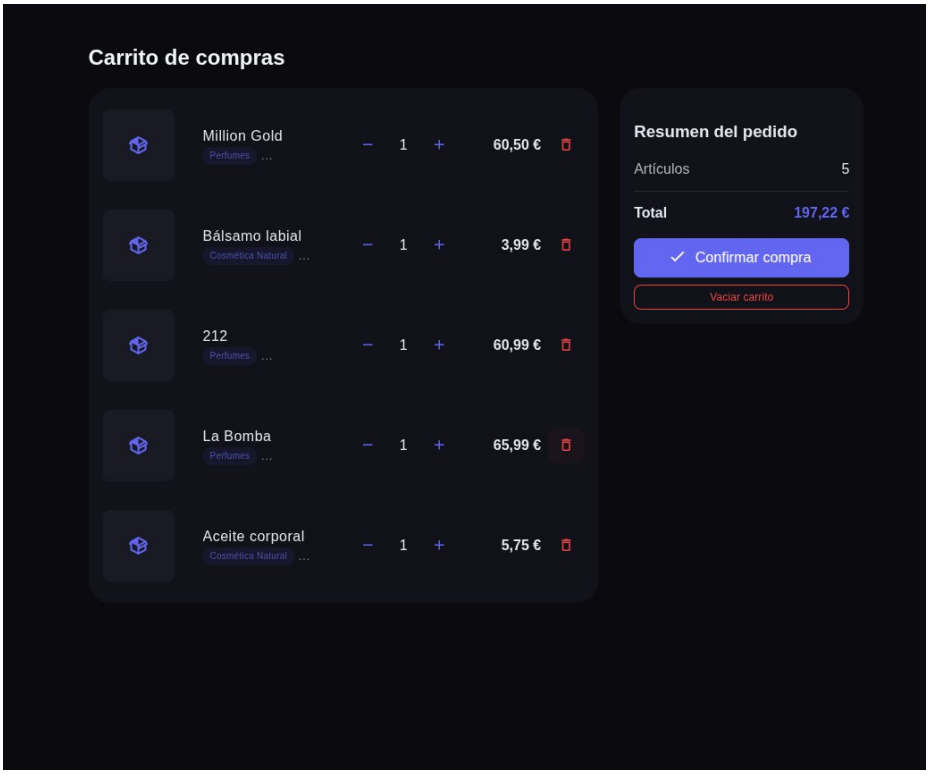



**Figura 5: Página carrito de compras.**

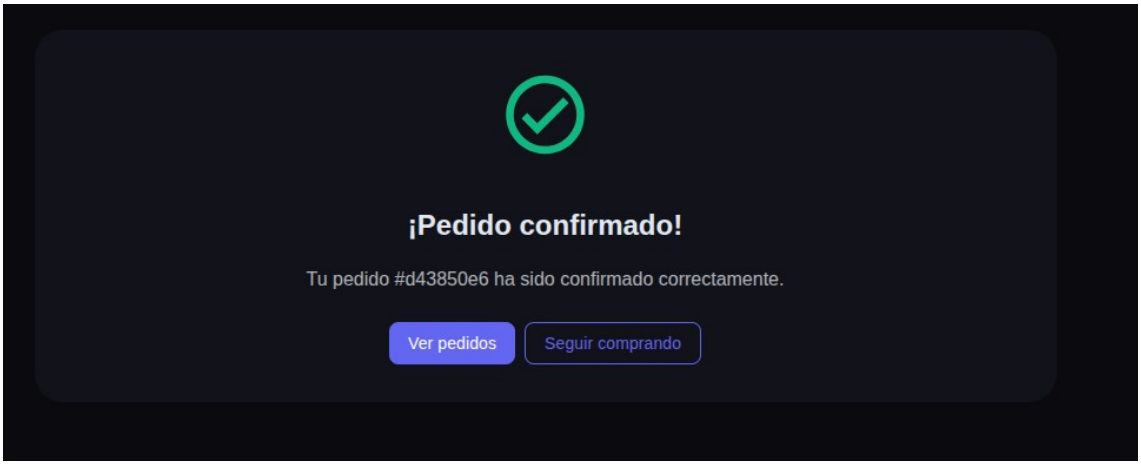



**Figura 6: Notificación de pedido confirmado.**

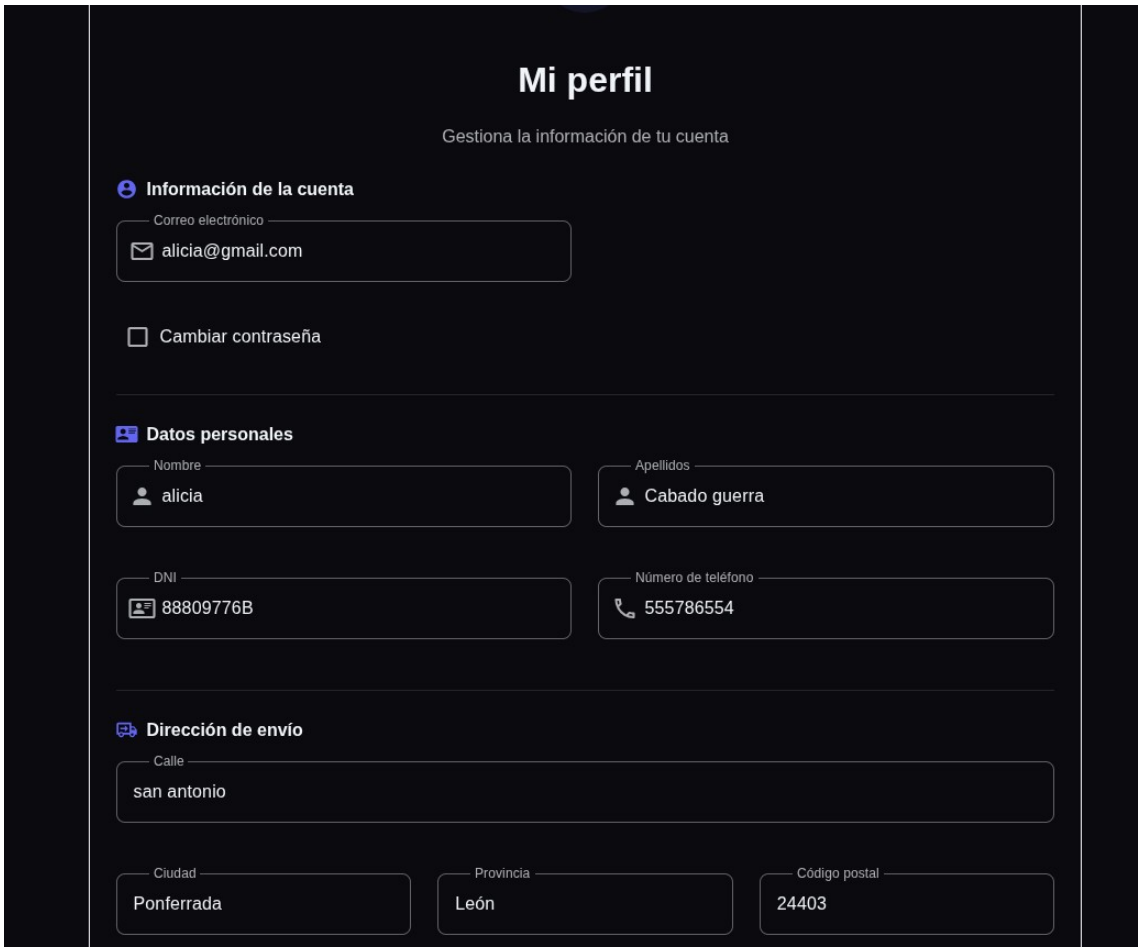



**Figura 7: Página de perfil de usuario.**

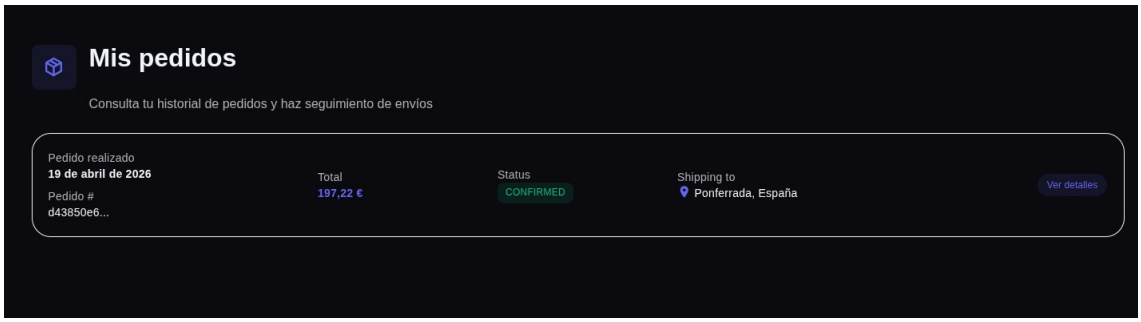



**Figura 8: Página de pedidos del usuario registrado.**

## 2.4. Concepción de la base de datos

**ARQUITECTURA DE BASE DE DATOS**

La base de datos del proyecto es una única instancia de PostgreSQL con CUATRO schemas separados, uno por módulo (bounded context):

```
PostgreSQL: maryshopping_postgres
├── Schema: customers
│   └── Tabla: customer
├── Schema: catalog
│   ├── Tabla: category
│   ├── Tabla: provider
│   ├── Tabla: product
│   └── Tabla: product_image
├── Schema: orders
│   ├── Tabla: purchase_order
│   └── Tabla: order_line
└── Schema: reports
    └── Tabla: order_report
```

Regla fundamental: NO existen claves foráneas entre schemas. Las referencias entre módulos se hacen únicamente por UUID (identificador lógico), sin FK declarada.

**DIAGRAMA MER**

```
CUSTOMER {
    UUID id PK
    string firstName
    string lastName
```

```
    string dni
    string phoneNumber
    string emailAddress
    string billingAddress_street
    string billingAddress_city
    string billingAddress_province
    string billingAddress_postalCode
    string billingAddress_country
    string shippingAddress_street
    string shippingAddress_city
    string shippingAddress_province
    string shippingAddress_postalCode
    string shippingAddress_country
}

CATEGORY {
    UUID id PK
    string name
}

PROVIDER {
    UUID id PK
    string name
}
```

```
PRODUCT {
    UUID id PK
    string name
    string description
    decimal price
    string productProviderRef
}

PRODUCT_IMAGE {
    UUID id PK
    string imageMimeType
    int imageOrder
    boolean isPrimary
    bytes imageBytes
}

PURCHASE_ORDER {
    UUID id PK
    date orderDate
    string status
    string shippingAddress_street
    string shippingAddress_city
    string shippingAddress_province
    string shippingAddress_postalCode
    string shippingAddress_country
```

```
}

ORDER_LINE {
    UUID id PK
    int quantity
    decimal unitPrice
}

ORDER_REPORT {
    UUID orderLineId PK
    date orderDate
    decimal subTotal
    decimal totalAmountDue
}

CUSTOMER → PURCHASE_ORDER : "realiza (1:N)"
PURCHASE_ORDER → ORDER_LINE : "contiene (1:N)"
PRODUCT → ORDER_LINE : "es referenciado en (1:N)"
CATEGORY → PRODUCT : "clasifica (1:N)"
PROVIDER → PRODUCT : "suministra (1:N)"
PRODUCT → PRODUCT_IMAGE : "tiene (1:N)"
PURCHASE_ORDER → ORDER_REPORT : "genera (1:N)"
ORDER_LINE → ORDER_REPORT : "detalla (1:1)"
```

## DIAGRAMA DE CLASES

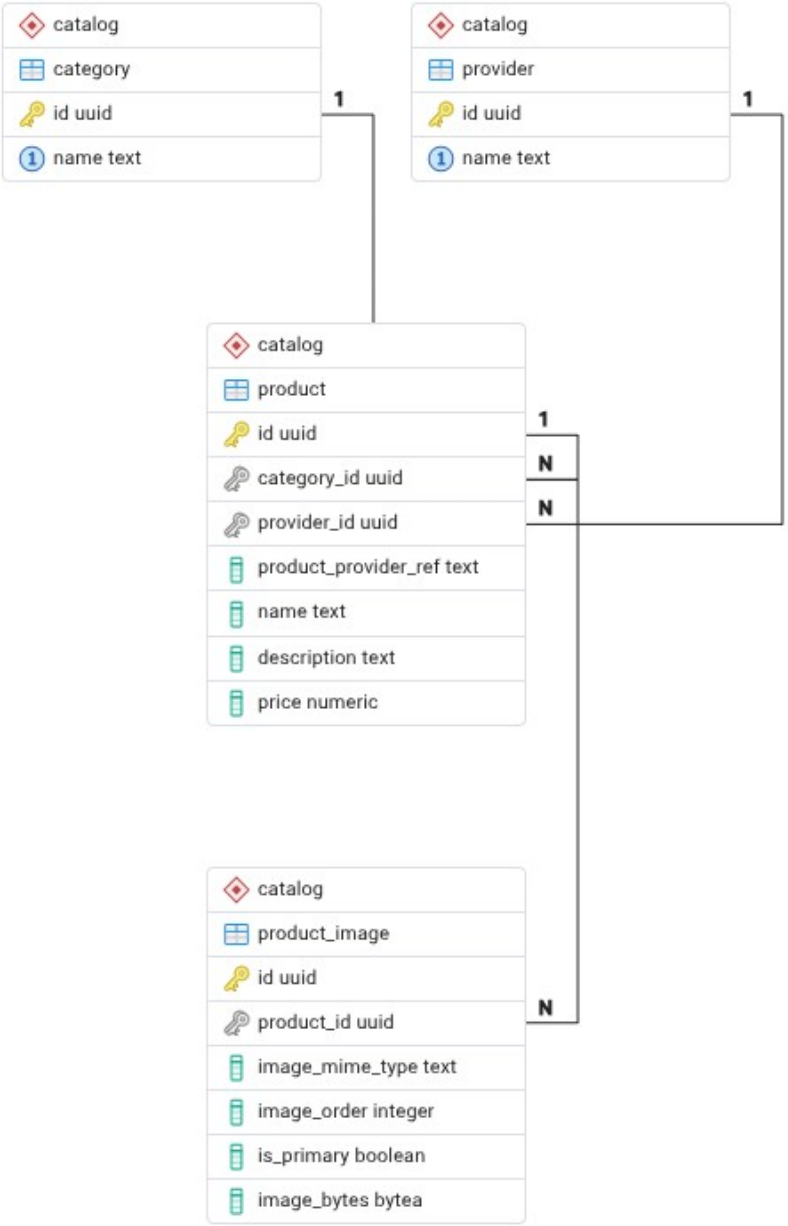



**Figura 9: Diagrama de clases de catálogo.**

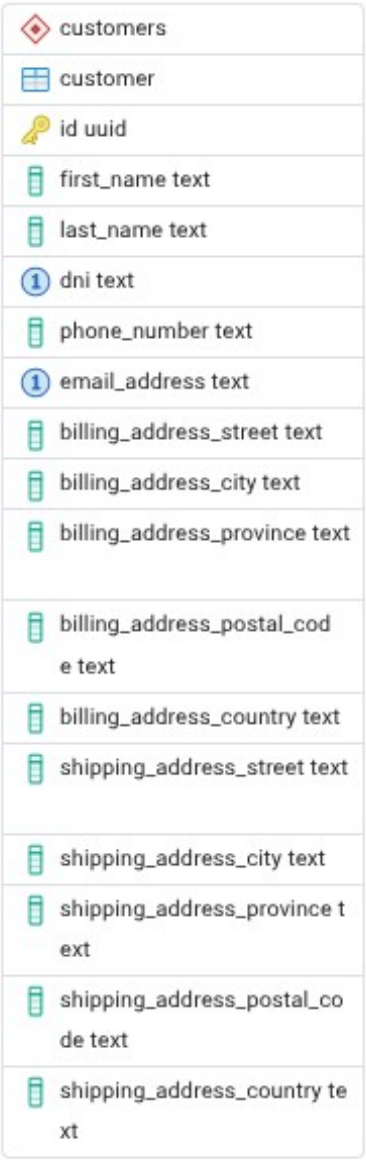



Figura 11: Diagrama de clases customers

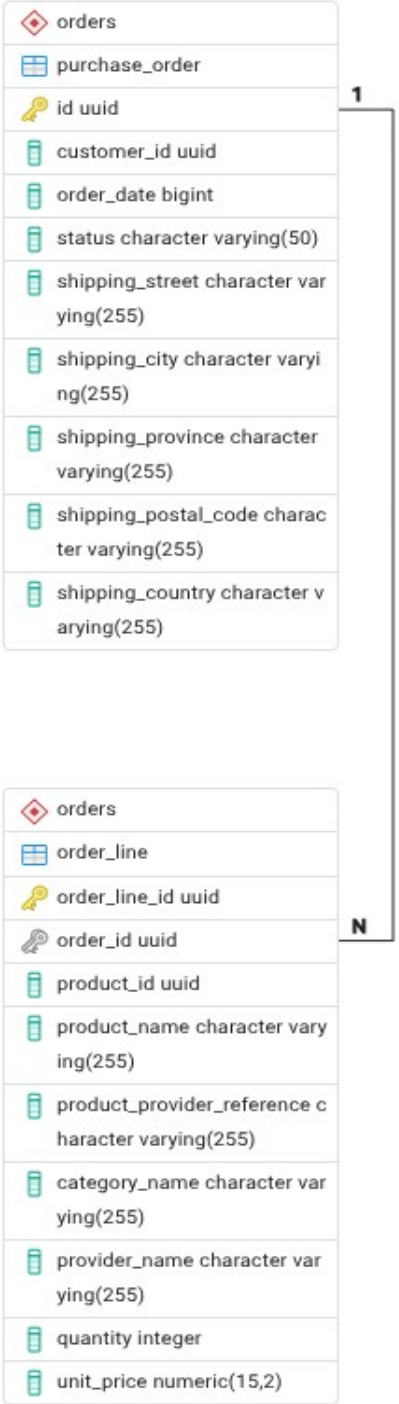



Figura 10: Diagrama de clases orders

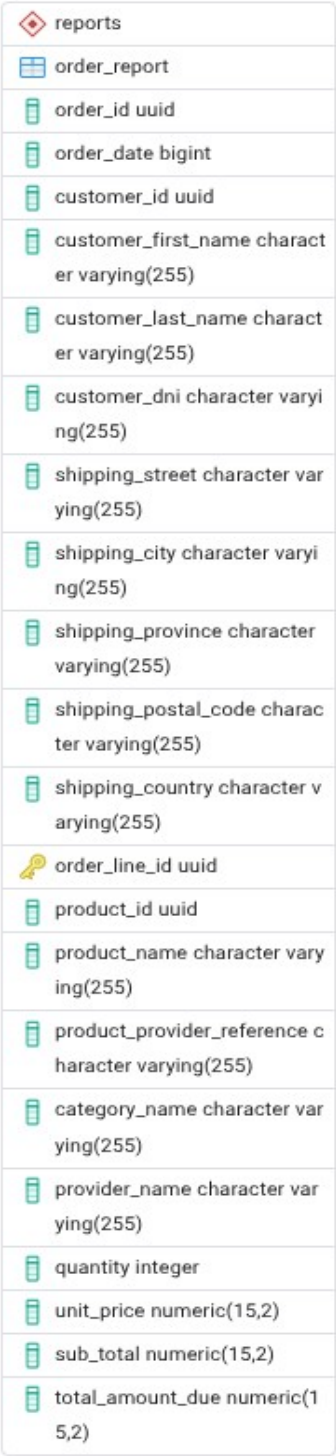



**Figura 12: Diagrama de clases reports**

## JUSTIFICACIÓN DEL DISEÑO DE BASE DE DATOS

El diseño de base de datos con schemas separados y sin FKs globales es una consecuencia directa de la arquitectura de módulos (bounded contexts) del proyecto. En resumen:

- La integridad referencial DENTRO de cada módulo sí se garantiza con FKs.

- La consistencia ENTRE módulos se garantiza en la capa de aplicación.

- Los schemas separados permiten que cada módulo evolucione de forma independiente.

- El módulo reports tiene su propio esquema optimizado para consultas de reporting, siguiendo el principio CQRS.

## GESTIÓN DE MIGRACIONES CON FLYWAY

Las migraciones de base de datos se gestionan con Flyway, con una configuración personalizada multi-módulo:

- Cada módulo tiene su propio conjunto de scripts SQL en:

  src/main/resources/{módulo}/db/migration/postgresql/

- Los scripts siguen la convención de nombres de Flyway:

  V{version}__{descripcion}.sql

  Ejemplo: V1_0_0__init_customers.sql

- FlywayConfig.java instancia un objeto Flyway por cada módulo, configurado para apuntar al schema correspondiente.

- MultiModuleFlywayMigrationStrategyConfig.java asegura que las migraciones de todos los módulos se ejecutan en el orden correcto al arrancar la aplicación.

## COMO SE ALIMENTA LA TABLA REPORTS

Cada vez que se confirma un pedido (caso de uso set_order_as_confirmed en el módulo orders), se dispara un evento (OrderConfirmedEvent) a través de la interfaz OrderConfirmed definida en el shared_kernel. El módulo reports implementa esa interfaz y recibe los datos del pedido confirmado, insertándolos en la tabla order_report como un snapshot inmutable.

## 2.5. Diseño y arquitectura de la solución

**VISIÓN GENERAL DE LA ARQUITECTURA DEL BACKEND**

EL backend aplica dos formas de organizar el código que se combinan entre sí:

**a)** MONOLITO MODULAR: una única aplicación (un solo fichero .jar, un solo despliegue) dividida por dentro en módulos bien separados. Es más sencillo que tener varios servicios separados (microservicios), pero aplica la misma idea de mantener cada parte independiente de las demás.

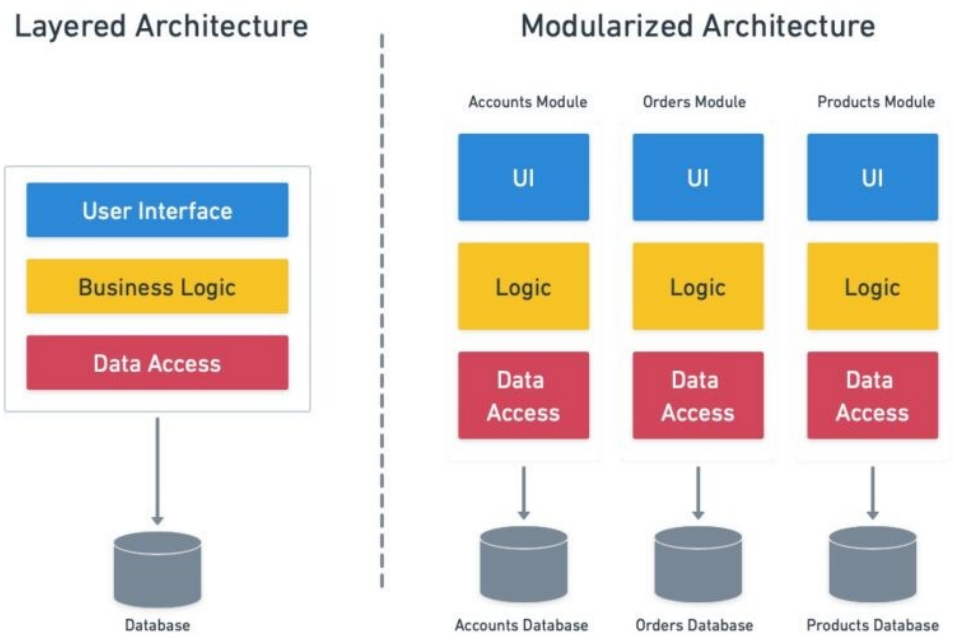



**Figura 13: Arquitectura por capas vs modular.**

**Fuente**: https://semaphore.io/blog/monolith-microservices

**b)** VERTICAL SLICE ARCHITECTURE: el código se organiza por caso de uso, no por capas técnicas (una carpeta para controllers, otra para services, etc…). Cada caso de uso tiene todo lo que necesita en su propia carpeta.

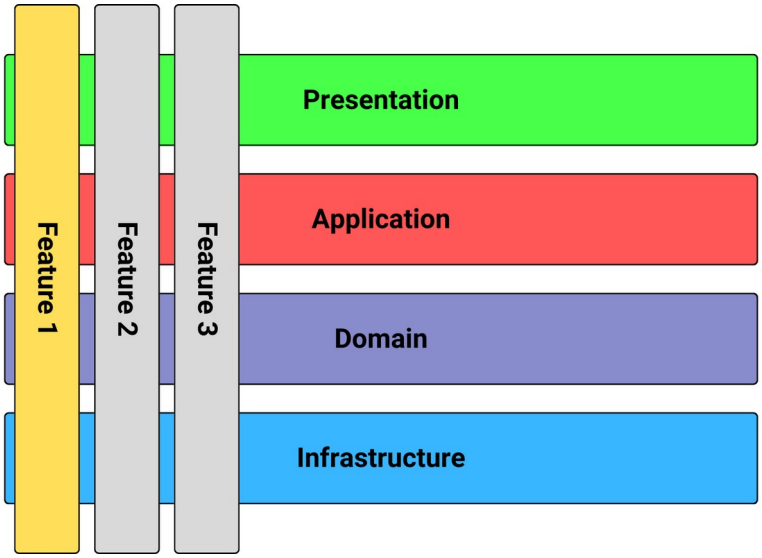



**Figura 14: Organización por casos de uso.**

**Fuente**: https://www.milanjovanovic.tech/blog/vertical-slice-architecture

Estos se complementan con:

- Clean Architecture y Domain-Driven Design (DDD): una forma de diseñar el código que pone en el centro las reglas del negocio (el dominio), en lugar de las tablas de la base de datos.

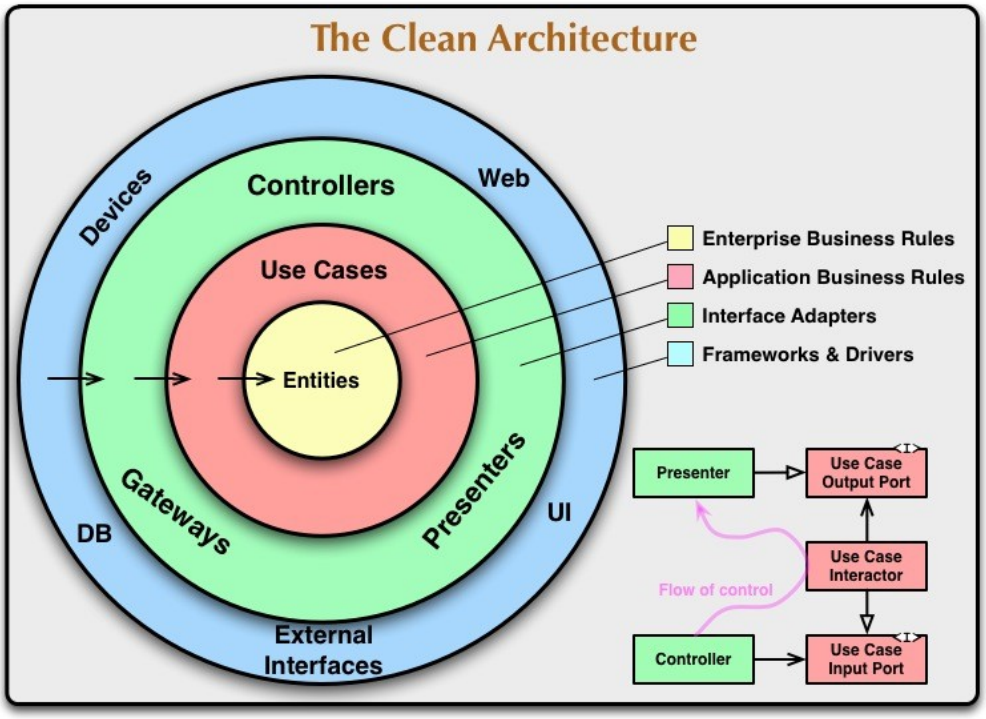



**Figura 15: Arquitectura limpia.**

**Fuente**: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

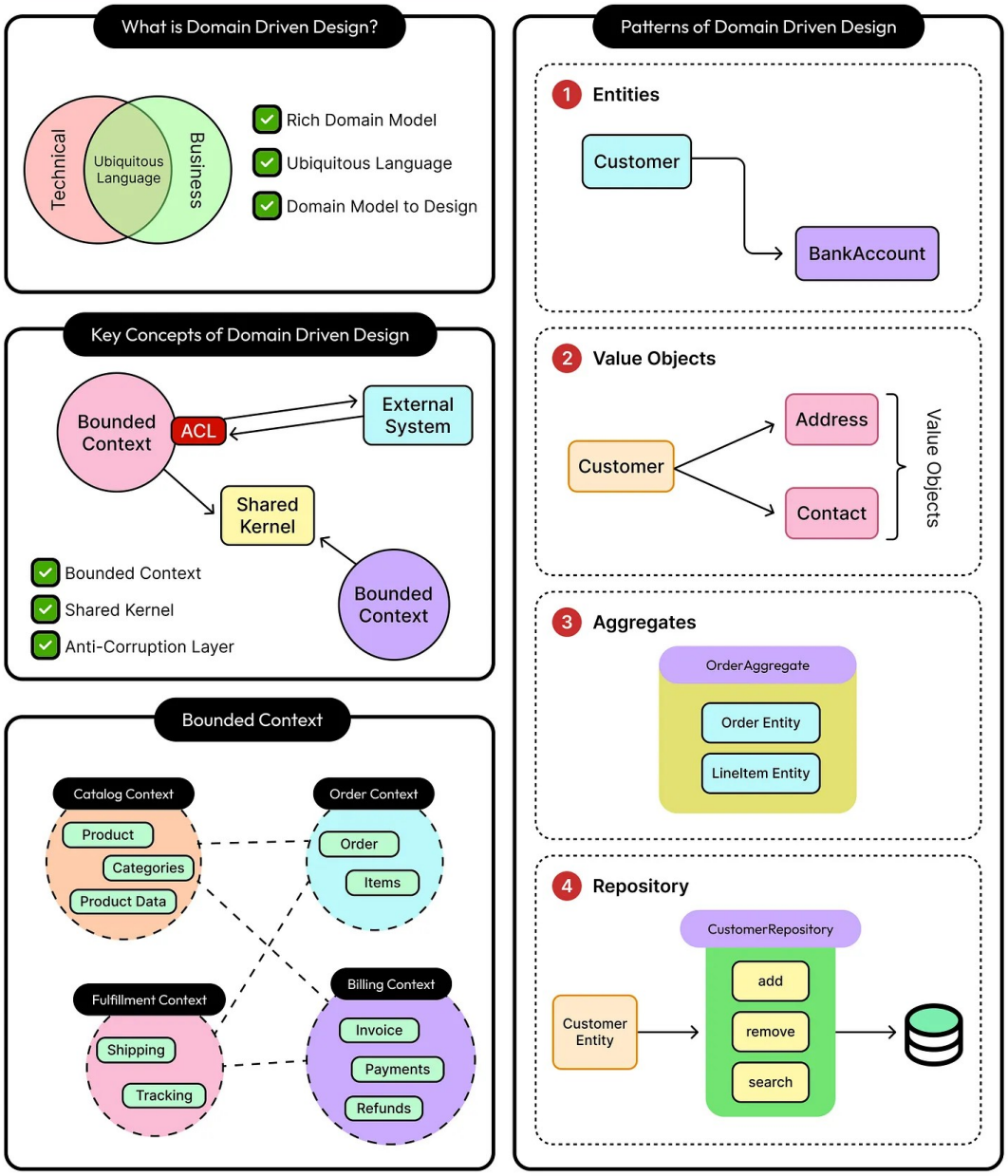



**Figura 16: DDD (Domain-Driven-Design)**

**Fuente**: https://blog.bytebytego.com/p/domain-driven-design-ddd-demystified

- CQRS parcial: el módulo reports separa el modelo de escritura (donde se guardan datos) del modelo de lectura (donde se consultan informes). "Parcial" significa que solo se aplica en reports; el resto de módulos usan la misma tabla para leer y escribir.

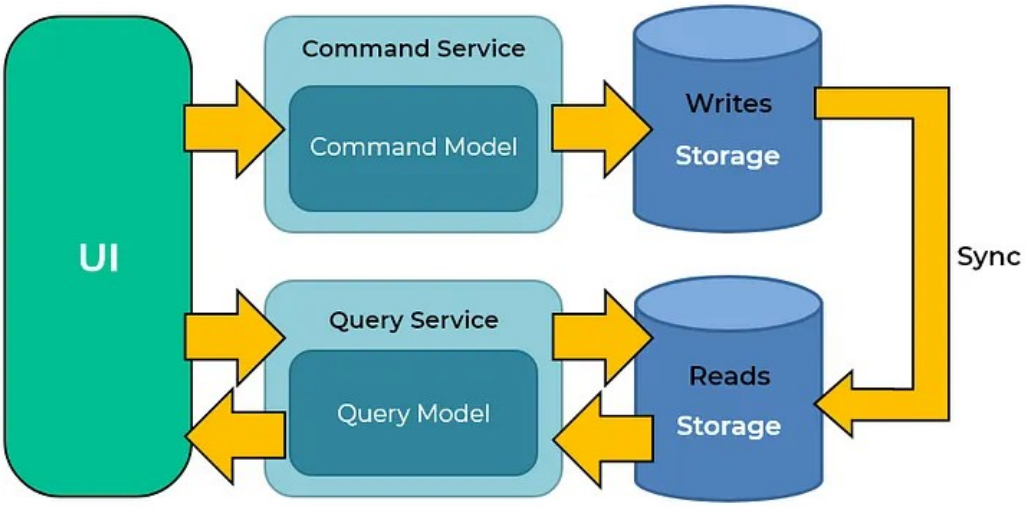



**Figura 17: CQRS**

**Fuente**: https://medium.com/@emer.kurbegovic/cqrs-software-architecture-pattern-the-good-the-bad-and-the-ugly-efe48e8dcd14

## ESTRUCTURA DE PAQUETES DEL BACKEND

La estructura de paquetes refleja la arquitectura:

```
src/main/java/es/maryshopping/backend/
├── BackendApplication.java                    ← Punto de entrada Spring Boot
│
├── configuration/                             ← Configuración centralizada de Spring
│   ├── FlywayConfig.java                      ← Configuración Flyway multi-módulo
│   ├── MultiModuleFlywayMigrationStrategyConfig.java
│   ├── SecurityConfig.java                    ← Filtros de seguridad y RBAC
│   ├── JwtDecoderConfig.java                  ←Decodificador JWT (Keycloak + blacklist)
│   └── KeycloakConfig.java                    ← Conexión con Keycloak Admin API
│
├── catalog/                                   ← Bounded context: catálogo
│   ├── shared/                                ← Entidad de dominio + repositorio compartido
│   │       ├── domain/                        ← Category, Product, Provider, Image...
│   |       └── infrastructure/persistence/    ← Entities JPA + Spring
 Repositories
│   └── usecases/
│       ├── create_category/
│                   ├── application/            ← CreateCategoryService,
CreateCategoryCommand, CreateCategoryResult
│       |           |__ infrastructure/rest/    ← CreateCategoryController,
 Request, Response
│       ├── create_product/                     (misma estructura)
│       ├── create_provider/                    (misma estructura)
│       ├── delete_category/                    (misma estructura)
│       ├── delete_product/                     (misma estructura)
│       ├── delete_provider/                    (misma estructura)
```

```
│       ├── get_all_categories/                    (misma estructura)
│       ├── get_all_providers/                     (misma estructura)
│       ├── get_category/                          (misma estructura)
│       ├── get_product/                           (misma estructura)
│       ├── get_products_in_category/              (misma estructura)
│       ├── get_provider/                          (misma estructura)
│       ├── images/
│       │   ├── upload_image/      ← Subida de imágenes (Multipart/form-data)
│       │   ├── get_image/                         ← Obtener una imagen por ID
│       │   ├── get_product_images/                ← Listar imágenes de un producto
│       │   ├── update_image/                      ← Actualizar metadatos de una imagen
│       │   └── delete_image/                      ← Eliminar una imagen
│       ├── update_category/                       (misma estructura)
│       ├── update_product/                        (misma estructura)
│       └── update_provider/                       (misma estructura)
│
├── customers/                                     ← Bounded context: clientes
│   ├── shared/
│   │   ├── domain/
                    |_ customer/        ← Customer, value objects (EmailAddress,
etc.)
|       |___infrastructure/
|   |           ├── persistence/ ← CustomerEntity, CustomerSpringRepository
│   │           |___iam/              ←    IAM  (interfaz),  KeycloakIAMService,
TokenIamResponse, LogoutIamResponse
│   └── usecases/
│       ├── create_customer/
│       ├── delete_customer/
│       ├── get_all_customers/
```

```
│       ├── get_customer/
│       ├── update_customer/
│       └── iam/                                  ← Casos de uso de autenticación
│           ├── get_token/                        ← Obtener token JWT (login)
│           └── logout/                           ← Cerrar sesión (revocar token)
│
├── orders/                                       ← Bounded context: pedidos
│   ├── shared/
│   │   ├── domain/              ← Order, OrderLine, StatusEnum, value objects
│       |____infrastructure/persistence/          ←OrderEntity,    OrderLineEntity,
OrderOwnershipResolver, Repositories
│   │
│   └── usecases/
│       ├── create_order/
│       ├── create_order_line/
│       ├── delete_order/
│       ├── delete_order_line/
│       ├── get_all_orders/
│       ├── get_order/
│       ├── set_order_as_confirmed/      ← Confirma un pedido (dispara evento)
│       ├── update_order/
│       └── update_order_line/
│
├── reports/                             ← Bounded context: reporting (sin dominio)
│   ├── shared/
│                   └──infrastructure/persistence/                ←OrderReportEntity,
OrderReportSpringRepository
│   └── usecases/
│       ├── get_top_customers/
```

│ ├── get_top_products/

│ |── save_order_confirmed/applicaation ← Recibe evento y guarda en order_report

│

└── shared_kernel/ ← Contratos compartidos entre módulos

│ └── domain/address/ ← Address, City, Street, Province, PostalCode, Country (value objects)

| ├── exceptions/ ← EntityNotFoundException, BusinessRuleException, DuplicatedEntityException, AuthenticationException, CustomerDataAccessForbiddenException, GlobalExceptionHandler

| ├── security/ ← CustomerOwnershipValidator, SecurityContextService, TokenBlacklistService

| └── internal_api/

| |__customer/

| |__application/ ← GetCustomerQuery, GetCustomerResult

| |__infrastructure/internal/ ← GetCustomer (interface)

| |__product/ (misma estructura)

| |__reports/ (misma estructura)

|

|__resources/ ← Scripts de migraciones Flyway y configuracion de la aplicación (application.yaml)

## ¿POR QUÉ SE ESCRIBE EL CÓDIGO EN INGLÉS?

Todo el código del proyecto (nombres de clases, variables, métodos, paquetes) está en inglés, aunque la documentación académica está en español. Las razones son:

- Es el estándar en la industria del software. Los tutoriales, las anotaciones de Spring, los mensajes de error y todas las librerías están en inglés. Escribir código mezclando idiomas (CreateCategoriaController, getProductoById) crearía confusiones.

- La letra ñ y los acentos causan problemas. Muchas herramientas (terminales, codificaciones de texto, URLs) no manejan bien los caracteres especiales del español. Un nombre de variable como "año" puede dar problemas de codificación.
- El software se desarrolla cada vez más en equipos internacionales. Usar inglés facilita que cualquier persona pueda leer y contribuir al código, independientemente de su idioma nativo.
- Toda la documentación técnica de referencia (Spring, Java, PostgreSQL) está en inglés. Usar los mismos términos en el código evita confusiones.

**PATRÓN VERTICAL SLICE — ESTRUCTURA DE UN CASO DE USO**

Cada caso de uso tiene tiene la misma estructura interna ( salvo en algunos casos (Delete, Get )que no llevan CreateCustomerRequest.

usecases/create_customer/

├── application/

│ ├── CreateCustomerService.java ← Lógica de flujo de negocio

│ └── CreateCustomerCommand.java ← DTO de entrada a la capa de aplicación

| |__CreateCustomerResult.java ← DTO de salida de la capa de aplicación

└── infrastructure/ rest/

| ├── CreateCustomerController.java ← Endpoint HTTP (REST)

| ├── CreateCustomerRequest.java ← DTO JSON de entrada HTTP

| └── CreateCustomerResponse.java ← DTO JSON de salida HTTP

**Flujo de una petición**:

1. El Controller recibe la petición HTTP con un Request (DTO JSON).

2. El Controller mapea el Request a un Command.

3. El Controller llama al Service con el Command.

4. El Service construye el objeto de dominio (validando y creando value objects).

5. El Service llama al Repository para persistir.

6. El Repository mapea el dominio a una entidad JPA y guarda en base de datos.

7. El Service devuelve un Result al Controller.

8. El Controller mapea el Result a una Response (DTO JSON) y responde.

¿POR QUÉ TANTOS OBJETOS PARECIDOS? Cada uno tiene una responsabilidad diferente y protege a su capa de los cambios en las demás.

**CAPAS Y RESPONSABILIDADES**

Capa de infraestructura (infrastructure/rest y shared/infrastructure/persistence):

/rest: dentro de cada caso de uso

- Controller: recibe la petición HTTP, delega en el Service y devuelve la respuesta HTTP.
- Request: record Java que representa el JSON de entrada.
- Response: record Java que representa el JSON de salida.

/persistence: compartida dentro del módulo(Bounded context)

- Entidad JPA: clase Java anotada con @Entity que mapea a la tabla de base de datos. Tiene un método estático fromDomain() para construirse desde el dominio.
- Spring Repository: interfaz que extiende JpaRepository y que Spring implementa automáticamente.

Capa de aplicación (application/): dentro de cada caso de uso

- Service: coordina la lógica del caso de uso. Construye el objeto de dominio, lo persiste a través del repositorio y devuelve el resultado.
- Command: contiene los datos necesarios para ejecutar el caso de uso. Es el “contrato” entre el controller y el service.
- Result: contiene solo los datos necesarios que el cliente espera recibir. No contiene lógica.

Capa de dominio (shared/domain): compartido dentro del módulo

- Entidad de dominio: record Java que representa el concepto del negocio con todos sus value objects. No tiene anotaciones JPA.
- Value objects: tipos que encapsulan un valor del dominio (City, Street, etc.).

En el caso de customers, en la infraestructura también temenos la implementación con /iam (para gestión de identidades y accesos), para los casos de uso de autenticación.

**PATRONES DE DISEÑO UTILIZADOS**

**a)** Builder Pattern (Patrón Constructor — Lombok @Builder):

Permite construir objetos paso a paso, indicando cada campo por su nombre. Esto evita confusiones cuando un objeto tiene muchos campos, porque no dependemos del orden de los parámetros.

**b)** Record (Java):

Un record es un tipo especial de clase que Java introdujo a partir de la versión 16. Es inmutable: una vez creado, no se puede cambiar. Java genera automáticamente los métodos equals(), hashCode() y toString(). Antes de los records, para un objeto con 5 campos había que escribir unas 40-50 líneas solo para el constructor, getters, equals y hashCode. Con un record es una sola línea:

```
public record City(String value) {}
```

Se usa para DTOs, Commands, Requests, Responses y Value Objects.

**c)** Repository Pattern (Patrón Repositorio):

Separa la lógica de negocio del acceso a datos. En el código, el repositorio se define como una interfaz (CustomerSpringRepository que extiende JpaRepository). Spring Data JPA genera la implementación automáticamente cuando la aplicación arranca: detecta que es una interfaz que extiende JpaRepository y crea internamente una clase que implementa todos los métodos heredados (findById, save, deleteById...). El programador nunca ve esa clase, pero existe y funciona.

**d)** Factory Method ( fromDomain()):

Las entidades JPA tienen un método estático fromDomain(Customer customer) que construye la entidad JPA a partir del objeto de dominio. Esto centraliza la lógica de conversión en un solo lugar.

**e)** Inyección de Dependencias por Constructor:

Este es uno de los conceptos más importantes de Spring y del proyecto. En lugar de que el servicio cree sus dependencias, alguien se las "da" (se las "inyecta") desde fuera. Ese "alguien" es Spring. El servicio solo dice "necesito un repositorio" (lo declara como parámetro del constructor), y Spring se encarga de crearlo y pasárselo. La documentación oficial de Spring recomienda la inyección por constructor.

**f)** Value Object (DDD):

Un Value Object es un tipo que encapsula un valor del dominio y se compara por su contenido, no por su posición en memoria. Dos objetos City ("Madrid") son iguales porque contienen el mismo valor, aunque sean dos objetos diferentes. En este proyecto, City, Street, PostalCode, Province, Country y Address son value objects.

**g)** Composición sobre herencia:

En programación orientada a objetos hay dos formas principales de reutilizar código(herencia y composición).Este proyecto usa composición casi exclusivamente. No hay jerarquías de herencia profundas. La herencia se usa solo donde Java o Spring la requieren (como extender JpaRepository). Esto hace que cada clase se pueda leer sin tener que buscar qué hereda de quién.

## EL MÓDULO REPORTS: PARTICULARIDADES

El módulo reports no sigue el patrón completo anterior. Al ser un módulo de solo lectura sin reglas de negocio complejas, se simplifican las capas:

- NO existe capa de dominio.
- El Service llama directamente al Repository con una consulta optimizada.
- El Repository devuelve proyecciones (objetos con solo los campos necesarios) con los datos agregados.
- La entidad JPA (OrderReportEntity) existe solo para la persistencia.

Esto aplica CQRS parcial: los módulos catalog, customers y orders están diseñados para crear y modificar datos; el módulo reports está diseñado para consultarlo de forma eficiente.

## SHARED KERNEL (NÚCLEO COMPARTIDO DE DDD)

En Domain-Driven Design, el Shared Kernel es un acuerdo entre varios módulos para compartir un conjunto pequeño y bien definido de código. En lugar de copiar los mismos tipos en cada módulo, se ponen en un sitio central que todos usan.

En este proyecto, el shared_kernel contiene:

- domain/address/ (record): value object que representa una dirección completa. Compuesto por: City, Country, PostalCode, Province, Street.
- exceptions/: excepciones comunes y el manejador global de errores: EntityNotFoundException, BusinessRuleException, DuplicatedEntityException, AuthenticationException, CustomerDataAccessForbiddenException, GlobalExceptionHandler.
- security/: servicios de seguridad compartidos entre módulos: CustomerOwnershipValidator (valida que un cliente solo accede a sus datos), SecurityContextService (extrae el usuario del token JWT), TokenBlacklistService (blacklist en memoria para logout inmediato).
- internal_api/: interfaces que permiten a los módulos comunicarse sin depender directamente unos de otros.

Es importante que el shared_kernel sea lo más pequeño posible: solo deben ir ahí los tipos que realmente necesitan compartirse.

## COMUNICACIÓN ENTRE MÓDULOS: INTERNAL_API

Los módulos no pueden acceder directamente al código de otros módulos. Para comunicarse, usan el shared_kernel/internal_api:

1. Se define una interfaz en el shared_kernel (GetCustomer, GetProduct, OrderConfirmed).
2. El módulo que “posee” los datos la implementa.
3. El módulo que necesita los datos depende SOLO de la interfaz, nunca del módulo concreto. Spring inyecta la implementación correcta.

Esto se basa en el Principio de Inversión de Dependencias (DIP): “Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones (interfaces).”

## COMANDOS , QUERIES Y EVENTOS

En el proyecto se usan tres tipos de objetos para comunicar datos entre capas:

- Command (Comando): Lleva los datos necesarios para una operación de escritura (crear, actualizar, eliminar). Los Commands NO contienen lógica. El mapeo a objetos de dominio se hace siempre en el Service (método fromCommandToDomain).
- Query (Consulta): Lleva los datos necesarios para una operación de lectura.
- Event (Evento): Lleva datos sobre algo que ya ha ocurrido. Se usan para avisar a otros módulos sin crear dependencias directas. Ejemplo: OrderConfirmedEvent contiene los datos del pedido confirmado. El módulo reports recibe este evento y crea los registros en su tabla de informes.

## GESTIÓN CENTRALIZADA DE ERRORES: GLOBALEXCEPTIONHANDLER

En lugar de manejar los errores en cada controller, el proyecto usa un manejador global con @RestControllerAdvice. Intercepta las excepciones de cualquier controller y las convierte en respuestas HTTP apropiadas:

- EntityNotFoundException → HTTP 400 (Bad Request)
- BusinessRuleException / DuplicatedEntityException → HTTP 409 (Conflict)
- Exception (cualquier otra) → HTTP 500 (Internal Server Error)
-AuthenticationException → HTTP 401 (Unauthorized)
-CustomerDataAccessForbiddenException → HTTP 403 (Forbidden)

## SUBIDA DE IMÁGENES: MULTIPART/FORM-DATA

El módulo catalog permite subir imágenes de productos. A diferencia de los demás endpoints que reciben JSON, este recibe multipart/form-data.

¿Qué es multipart/form-data? Cuando un usuario sube un fichero a una web (una foto, un PDF), el navegador no puede enviarlo como texto JSON. El fichero son bytes (datos binarios). Para enviar estos bytes se usa

multipart/form-data, que divide el contenido en “partes”: una con datos de texto y otra con el fichero. Spring Boot las recibe automáticamente como un objeto MultipartFile.

¿Por qué no enviar la imagen dentro del JSON? Se podría convertir la imagen a texto usando Base64 (una codificación que transforma bytes en caracteres legibles) y meterla en un JSON ,pero Base64 aumenta el tamaño de los datos aproximadamente un 33%. En una aplicación con muchas imágenes, eso es mucho tráfico de red desperdiciado.

Además, el JSON tiene que cargarse entero en memoria antes de procesarse. Con multipart/form-data, el servidor puede procesar el fichero a medida que lo recibe (en trozos), usando menos memoria.

En el proyecto, el UploadImageController:

- Recibe la imagen como MultipartFile (los bytes).
- Recibe metadatos (orden, si es imagen principal, ID del producto).
- Extrae los bytes (imageFile.getBytes()) y el tipo MIME (imageFile.getContentType(), por ejemplo “image/jpeg”).
- Los almacena en la base de datos como campo BYTEA en product_image.

**CONFIGURACIÓN FLYWAY MULTI-MÓDULO**

La configuración estándar de Flyway asume una única fuente de migraciones. Este proyecto necesita cuatro instancias independientes (una por módulo), cada una apuntando a su propio esquema PostgreSQL.

- Se activa Flyway en application.yaml (spring.flyway.enabled: true) pero se complementa con una configuración personalizada multi-módulo.

- Se define una sección personalizada en application.yaml (flywayconf) con los schemas y locations de cada módulo.

- FlywayConfig.java lee esta configuración y la expone como beans de Spring.

- MultiModuleFlywayMigrationStrategyConfig.java crea una instancia de Flyway por módulo usando la API de Flyway.

¿Cómo funciona Flyway internamente?

1. Al arrancar, Flyway se conecta a la base de datos.
2. Busca una tabla interna (flyway_schema_history) donde registra qué scripts se han ejecutado ya.
3. Compara los scripts disponibles con los ya ejecutados.
4. Ejecuta los pendientes en orden de versión.
5. Si un script falla, se detiene para no dejar la base de datos a medias.

## FLUJO PRINCIPAL DE LA APLICACIÓN

El flujo de negocio de MarySHopping:

1. El administrador crea proveedores (POST /catalog/products/providers).
2. El administrador crea categorías (POST /catalog/categories).
3. El administrador crea productos asociados a categoría y proveedor (POST /catalog/products).
4. El administrador sube imágenes (POST /catalog/products/images). El catálogo está completo y visible.
5. Los clientes ven el catálogo (GET /catalog/products).
6. Un cliente se registra (POST /customers) y obtiene token (POST /customers/iam/token).
7. El cliente crea un pedido (POST /orders/order).
8. El cliente añade productos al pedido (POST /orders/orderLine).
9. Se confirma el pedido (PUT /orders/order/status/confirmed/{id}). El servicio cambia el estado a CONFIRMED y avisa al módulo reports a través del shared_kernel. Se alimenta la tabla de informes.
10. El administrador consulta informes: Top productos más vendidos (GET /reports/top-products). Top clientes por volumen (GET /reports/top-customers).

## JUSTIFICACIÓN DEL USO DE UUID

Todas las claves primarias son UUIDs generados por la aplicación, en lugar de IDs autoincrementales generados por la base de datos (id=1, id=2, id=3...).

¿Por qué?

**a)** El objeto existe antes de guardarse: Con un UUID, el objeto tiene su ID desde que se crea en el código, antes de ir a la base de datos. Con un ID autoincremental, hay que esperar al INSERT para que la base de datos asigne el número.

**b)** No hay colisiones entre instancias: Si hay varias copias del servidor funcionando a la vez, un ID autoincremental podría repetirse. Un UUID es único sin necesidad de coordinación.

**c)** Seguridad: Los IDs autoincrementales son predecibles. Los UUIDs no son predecibles.

**d)** Coherencia con DDD: El ID es propiedad del objeto de dominio, no de la base de datos.

**JUSTIFICACIÓN DEL USO DE LONG PARA FECHAS**

Las fechas se almacenan como BIGINT (Long en Java): milisegundos desde el 1 de enero de 1970 UTC.

a) Sin problemas de zona horaria: un número siempre significa lo mismo.

b) Compatible con JavaScript: Date.now() devuelve milisegundos.

c) Comparar fechas es comparar números: fecha1 < fecha2.

d) Funciona igual en todas las bases de datos.

**SNAPSHOTS: COPIAS INMUTABLES DE DATOS**

Un snapshot es una copia de datos que se guarda tal como eran en un momento concreto.

Ejemplo:

Un cliente pide la "Camiseta Azul" a 19,99 €. Dos semanas después, el precio sube a 24,99 €. El pedido del cliente debe seguir mostrando 19,99 €.

**REDUNDANCIA DELIBERADA DE DATOS EN DDD**

El mismo dato puede existir en varios sitios:

- catalog.product → nombre actual (editable).

- orders.order_line → nombre cuando se hizo el pedido (snapshot).

- reports.order_report → nombre en el informe (snapshot).

En DDD, esto es una decisión consciente. Cada módulo tiene su propia visión del dato y es completamente independiente.

## PROGRAMACIÓN FUNCIONAL EN JAVA: LAMBDAS, METHOD REFERENCES Y STREAMS

Java, a partir de la versión 8, incorporó herramientas de programación funcional que se usan en este proyecto con frecuencia.

Una lambda es una función anónima: una función sin nombre que se escribe directamente donde se necesita, en lugar de crear un método separado.

Ejemplo: List activos = productos.stream() .filter(p -> p.isActive()) .toList();

Un method reference es una forma más corta de escribir una lambda cuando esta simplemente llama a un método ya exixtente.

Ejemplo: filter(Product::isActive).

Un Stream es una forma de procesar listas de datos indicando qué hacer con ellos, en lugar de cómo hacerlo paso a paso.

Ejemplo: List results = entities.stream() .map(entity -> GetOrderResult.fromEntity(entity)) .toList();

## PREPARACIÓN PARA CLOUD Y ESCALABILIDAD

Aunque el backend es un monolito modular, las decisiones de diseño están pensadas para que pueda crecer:

**a)** Stateless (sin estado en el servidor).

En este proyecto, se usa el JWT (JSON Web Token): un token que el cliente envía en cada petición HTTP (en la cabecera Authorization). El token contiene toda la información necesaria para identificar al usuario y sus permisos. El servidor no guarda nada sobre el usuario entre peticiones: no hay “sesiones” en el servidor. Esto es importante porque permite poner varias copias del servidor funcionando a la vez. Como ninguna guarda estado, cualquiera puede atender cualquier petición. Si una se cae, las demás siguen funcionando.

**b)** Escalado vertical vs escalado horizontal:

- Escalado vertical: poner una máquina más potente (más RAM, más CPU).

- Escalado horizontal: poner más máquinas iguales en paralelo.

El diseño sin estado permite el escalado horizontal: basta con ejecutar más copias del backend en Kubernetes. Con un diseño “con estado” (sesiones en el servidor), las copias tendrían que compartir la información de sesión entre ellas, lo que es mucho más complejo.

**c)** Modularidad como preparación: Si en el futuro fuera necesario extraer un módulo a un servicio independiente, los límites ya están definidos.

**d)** UUIDs: permiten generar IDs sin coordinación.

**e)** Fechas epoch: eliminan problemas de zona horaria.

**CONVENCIÓN DE NOMBRES: SNAKE_CASE EN PAQUETES**

Los paquetes de los casos de uso usan snake_case (create_customer,get_all_orders). La convención habitual en Java es minúsculas sin separadores, pero la especificación no prohíbe guiones bajos. Se usa por legibilidad: “create_customer” es más claro que “createcustomer”.

**ARQUITECTURA DESACOPLADA: FRONTEND Y BACKEND**

La solución global aplica una arquitectura desacoplada en la que el frontend y el backend son dos aplicaciones completamente independientes que se comunican exclusivamente a través de una API REST.

El backend (Spring Boot) expone endpoints HTTP que devuelven y reciben JSON. No genera vistas ni HTML: su única responsabilidad es procesar la lógica de negocio y responder datos estructurados.

El frontend (Nuxt 4 con Vue 3 y Vuetify 4) consume esos endpoints y es el único responsable de la presentación visual. No contiene lógica de negocio: simplemente muestra los datos que el backend le devuelve y le envía las acciones del usuario.

Este desacoplamiento aporta varias ventajas:

Desarrollo independiente: el equipo de frontend y el de backend pueden trabajar en paralelo sin bloquearse, siempre que respeten el contrato de la API.

Tecnología libre en cada capa: el frontend podría reemplazarse por una aplicación móvil u otro framework web sin tocar una sola línea del backend.

Separación clara de responsabilidades: el backend no sabe cómo se muestra la información; el frontend no sabe cómo se procesa.

Escalabilidad independiente: frontend y backend se despliegan y escalan por separado en Kubernetes.

Comunicación entre ambos:

El navegador pide la página web. Kubernetes sirve el frontend (html, css, js estático). El JS se descarga y se ejecuta en el navegador. A partir de ahí es una SPA. Vue necesita datos, Axios hace la petición, el ingress ve que la ruta empieza por /maryshopping y lo manda al pod del backend. Spring Boot responde con un JSON y Vue renderiza la pantalla con esos datos.

## 2.6. Principales dificultades encontradas y cómo se han afrontado.

Las dificultades encontradas han sido muchas , pero de todas ellas las mas destacadas son:

**CONFIGURACIÓN MULTI-MÓDULO DE FLYWAY**

**DIFICULTAD**: Flyway, por defecto, gestiona una única fuente de migraciones apuntando a un único schema. En este proyecto hay cuatro schemas independientes (catalog, customers, orders, reports), cada uno con sus propias migraciones. La configuración estándar de Spring Boot no soporta esto directamente.

**SOLUCIÓN ADOPTADA**: Se desarrolló una configuración personalizada con dos clases:

FlywayConfig.java: instancia manualmente cuatro objetos Flyway, uno por módulo, cada uno configurado con su schema y su ruta de scripts de migración.

MultiModuleFlywayMigrationStrategyConfig.java: anula la estrategia de migración por defecto de Spring Boot para ejecutar las cuatro instancias de Flyway en el orden correcto al arrancar la aplicación.

## ENTENDER Y APLICAR DDD POR PRIMERA VEZ

**DIFICULTAD**: Domain-Driven Design es un concepto complejo con muchos términos y patrones nuevos (bounded context, value object, repository,..). Al empezar el proyecto, la cantidad de conceptos fue abrumadora.

**SOLUCIÓN ADOPTADA**: Se optó por una aproximación práctica: empezar por implementar el caso de uso más sencillo (create_category) y repetir ese patrón para todos los demás. Una vez que el primer caso de uso funcionó correctamente, el resto fue aplicar la misma estructura. La clave fue entender la diferencia entre: El objeto de dominio( representa el concepto del negocio). La entidad JPA (representa cómo se persiste ese concepto en la base de datos) y separar estas dos responsabilidades en clases distintas es la esencia del DDD en este proyecto.

## ADDRESS COMO VALUE OBJECT COMPARTIDO

**DIFICULTAD**: El concepto “dirección” (Address) aparece en varios módulos: los clientes tienen dirección de facturación y envío, los pedidos tienen dirección de envío. ¿Cómo compartir este tipo sin crear acoplamiento entre módulos?

**SOLUCIÓN ADOPTADA**: Se creó el shared_kernel que contiene tipos compartidos. Address y sus value objects (City, Street, Province, PostalCode, Country) se definen en shared_kernel.domain.address y pueden ser usados por cualquier módulo.

En la base de datos, Address no tiene tabla propia: se almacena como columnas embebidas en la tabla de cada entidad que la usa (billing_address_street, billing_address_city, etc.). Esto evita joins innecesarios.

## DEFINIR LOS LÍMITES ENTRE MÓDULOS

**DIFICULTAD**: ¿Dónde termina un módulo y dónde empieza otro? ¿Qué pasa cuando un pedido necesita datos de un cliente o de un producto?

**SOLUCIÓN ADOPTADA**: Se adoptó el patrón de snapshot: cuando se crea un pedido, los datos relevantes del cliente y del producto (nombre, referencia, precio…) se copian en la tabla orders como texto plano. Así, el módulo orders no necesita consultar customers o catalog para mostrar la información histórica de un pedido.

Esta decisión tiene un coste: si el nombre de un producto cambia, los pedidos antiguos mantienen el nombre original. Pero esto es, de hecho, el

comportamiento correcto para un sistema de e-commerce: el pedido debe reflejar los datos en el momento de la compra, no los datos actuales.

**MÓDULO REPORTS: QUERIES AGREGADAS CON SPRING DATA JPA**

**DIFICULTAD**: Los casos de uso de reporting (top_products, top_customers) requieren queries SQL con GROUP BY, SUM y ORDER BY sobre la tabla reports.order_report. Spring Data JPA no genera estas queries tan complejas automáticamente a partir del nombre del método.

**SOLUCIÓN ADOPTADA**: Uso de la anotación @Query de Spring Data JPA con JPQL para definir las consultas directamente en la interfaz del repositorio.

**INTEGRACIÓN CON KEYCLOAK: AUTENTICACIÓN Y GESTIÓN DE USUARIOS**

**DIFICULTAD**: Entender los flujos OAuth2/OpenID Connect, integrar la Admin API de Keycloak para gestionar usuarios desde el backend, y conseguir un logout real e inmediato.

**SOLUCIÓN ADOPTADA**: Se configuró Spring Security con oauth2-resource-server para validar tokens JWT usando la clave pública del realm (jwks-uri). Se creó un cliente dedicado con service_account para las llamadas a la Admin API, encapsuladas en KeycloakIAMService. Para el logout, se implementó una TokenBlacklistService en memoria: al cerrar sesión, el token se revoca en Keycloak y se añade a la blacklist local, de modo que queda invalidado de inmediato aunque el JWT aún no haya caducado.

**SEGURIDAD: SPRING SECURITY Y RBAC**

**DIFICULTAD**: Los roles de Keycloak llegan en el JWT dentro de una estructura anidada (realm_access.roles) que Spring Security no interpreta directamente. Además, los endpoints de catálogo deben ser accesibles sin token, pero Spring rechazaba con 401 cualquier token inválido aunque el endpoint fuera público.

**SOLUCIÓN ADOPTADA**: Se implementó un JwtAuthenticationConverter personalizado que extrae los roles del campo realm_access.roles y los convierte en authorities de Spring Security. Toda la configuración de acceso por rol se centralizó en SecurityConfig.java. Los endpoints públicos se configuraron con permitAll() y con el bearer token resolver en modo tolerante, permitiendo peticiones sin token y con token válido por igual.

## DESPLIEGUE EN KUBERNETES

**DIFICULTAD**: Gestionar la comunicación entre Pods (backend, PostgreSQL, Keycloak), la configuración diferente entre entorno local y Kubernetes, y la persistencia de datos de PostgreSQL al reiniciar Pods.

**SOLUCIÓN ADOPTADA**: Se usaron nombres de Service de Kubernetes para la comunicación entre Pods (DNS interno del clúster). La configuración se externaliza mediante ConfigMaps y Secrets, leídos como variables de entorno en application.yaml. El backend se expone con un Service NodePort y PostgreSQL monta sus datos en un PersistentVolumeClaim para sobrevivir a reinicios.

## AISLAMIENTO REAL ENTRE BOUNDED CONTEXTS

**DIFICULTAD**: Conseguir que los bounded contexts sean verdaderamente independientes es más difícil de lo que parece. La tentación permanente es crear dependencias entre módulos.

**SOLUCIÓN ADOPTADA**: Se siguieron estrictamente los principios de DDD para el aislamiento entre módulos (Snapshots para datos que cruzan fronteras de módulos, Interfaces en shared_kernel para comunicación controlada, disciplina de no importar código de otros módulos y sin claves foráneas entre schemas).

## MÉTODOS QUE SIEMPRE DEVUELVEN UN VALOR

**DIFICULTAD**: En muchos tutoriales y ejemplos de Spring Boot, los servicios devuelven void y usan excepciones como única forma de indicar errores. Esto hace difícil seguir el flujo de los datos a lo largo del código.

**SOLUCIÓN ADOPTADA**: Todos los métodos de servicio del proyecto devuelven un valor (nunca void). Cada operación produce un resultado que la capa que la invoca puede comprobar. Los métodos de mapeo (fromCommandToDomain, fromDomain, mapFromRequestToCommand) son funciones puras (reciben datos de entrada y devuelven un nuevo objeto sin modificar nada externo).

## BLOQUEO DE PETICIONES POR PARTE DEL NAVEGADOR (CORS)

**DIFICULTAD**: Cuando se hacía una petición al backend, el navegador bloqueaba la respuesta (no dejaba verla).Esto se debe a una regla de seguridad de los navegadores llamada SOP (política del mismo origen) para proteger al usuario.

**SOLUCIÓN ADOPTADA**: Desactivar los CORS ( mecanismo de defensa que permite al backend autorizar peticiones de otros orígenes añadiendo ciertas cabeceras en sus respuestas HTTP), ya que esto es un proyecto de fin de ciclo y no una aplicación en produccion real. En producción real , esto sería un riesgo de seguridad.

## 2.7. Despliegue (dónde está alojada, cómo acceder (usuarios, contraseñas), qué se necesita, etc.)

El proyecto cuenta con dos entornos de despliegue:

- Desarrollo (deployment/development/): Docker Compose para levantar PostgreSQL en local de forma rápida. El frontend y el backend se arrancan directamente con sus herramientas de desarrollo (pnpm dev y ./mvnw spring-boot:run o IntelliJ).
- Producción (deployment/production/): MicroK8s (Kubernetes) con todos los componentes: PostgreSQL, Keycloak, backend y frontend.

**ESTRUCTURA DEL MONOREPO**

El proyecto se organiza como un monorepo con tres carpetas principales:

```
maryshopping/
├── backend/
│   ├── src/
│   ├── pom.xml
│   ├── Dockerfile
│   └── postman/
├── frontend/
│   ├── app/
│   ├── nuxt.config.ts
│   └── package.json
└── deployment/
```

```
├── development/                    ← Docker Compose (desarrollo local)
│   └── postgres/
└── production/                     ← Kubernetes (MicroK8s)
    ├── namespaces/
    ├── infrastructure/
    │   ├── postgres/
    │   └── keycloak/
    ├── apps/
    │   ├── backend/
    │   └── frontend/
    ├── confidential/               ← (.gitignore'd) credenciales GHCR
    └── documentation/
```

**REQUISITOS PARA EL DESPLIEGUE EN PRODUCCIÓN**

**Cluster de Kubernetes (microk8s)**:

- MicroK8s instalado (sudo snap install microk8s –classic).

- Addons activados: dns, storage, ingress.

**Backend (app)**:

- Docker instalado en la máquina.

- Java 25 (JDK) para compilar el proyecto con Maven.

- Maven instalado.

- Acceso a GitHub Container Registry (GHCR) para publicar la imagen.

- Acceso a una instancia de PostgreSQL con:

Host: maryshopping-postgres.postgres.svc.cluster.local (K8s)

Puerto: 5432 (interno) / 31543 (NodePort externo)

Base de datos: maryshopping_postgres

Usuario: postgres

Contraseña: postgres

**Frontend (app)**:

- Node.js 22 LTS (instalable vía nvm).

- pnpm 10.x (gestor de paquetes, via corepack).

- Docker instalado (para construir y publicar la imagen en producción).

- Acceso a GitHub Container Registry (GHCR) para publicar la imagen.

**DIFERENCIA ENTRE DESPLEGAR INFRAESTRUCTURA VS APPS**

Para PostgreSQL y Keycloak el proceso es mas sencillo. Kubernetes descarga las imágenes oficiales directamente de Docker Hub (tú no construyes nada, las imágenes ya existen y son públicas). En cambio en las apps, el código es propio, con lo cual se debe construir la imagen y publicarla en GHCR. Para ello hay que crear un fichero llamado Dockerfile, registrarse en GHCR y guardar las credenciales en mi caso en (.env, si la imagen es privada) para que posteriormente kubernetes pueda descargarla.

**CÓMO COMPILAR LA APLICACIÓN Y GENERAR Y SUBIR LAS IMAGENES**

Antes de compilar se debe:

- Ir al directorio deployment

  cd /ruta/a/maryshopping/deployment

- Cargar credenciales desde el fichero .env

  source production/confidential/.env

- *Preparar Docker para esta sesión (solo necesario una vez por terminal)

  export DOCKER_CONFIG=$(mktemp -d)

  export DOCKER_HOST=unix://$HOME/.docker/desktop/docker.sock

- Autenticarse en GHCR

  echo $GITHUB_TOKEN | docker login ghcr.io -u $GITHUB_USER –password-stdin

- Calcular la etiqueta de versión (desde deployment/, donde está el repositorio git)

  TAG=$(date +%Y%m%d)-$(git rev-parse --short HEAD)

* Es necesario en mi caso al utilizar Docker Desktop.

En el caso del:

**BACKEND**

- Ir al directorio del backend y compilar el proyecto con Maven

  cd /ruta/a/maryshopping/backend

  ./mvnw package -DskipTests

- Desempaquetar el JAR en capas (lo requiere el Dockerfile)

  mkdir -p target/dependency && (cd target/dependency; jar -xf ../*.jar)

- Construir la imagen Docker

  docker build -t ghcr.io/$GITHUB_USER/maryshopping/maryshopping-backend:$TAG .

- Subir la imagen a GHCR

  docker push ghcr.io/$GITHUB_USER/maryshopping/maryshopping-backend:$TAG

**FRONTEND**

El flujo es identico al del backend, pero con algunas diferencias importantes en cada paso:

- El dockerfile usa el patrón multi-stage build para mantener la imagen final pequeña (solo contiene lo mínimo para ejecutar la aplicación).
- Las credenciales y preparación de Docker son las mismas que para el backend (el mismo fichero .env).
- Una vez realizados los mismos pasos hasta calcular la etiqueta de versión, incluida. Los paso serían:
- Ir al directorio del frontend

cd /home/marisol/ciclo_fp_desarrollo_de_aplicaciones_web/proyecto_DAW/maryshopping/frontend

- construir la imagen Docker

  docker build -t ghcr.io/$GITHUB_USER/maryshopping/maryshopping-frontend:$TAG .

- Subir la imagen a GHCR

  docker push ghcr.io/$GITHUB_USER/maryshopping/maryshopping-frontend:$TAG

**DESPLIEGUE EN KUBERNETES (PRODUCCIÓN)**

Todos los recursos de kubernetes (Deployment, PVC, service, Ingress, etc...) que se usan para el despliegue se explican en la webgrafía del documento.

**PostgreSQL**

Desplegamos PostgreSQL como un Deployment estándar de Kubernetes. Le ponemos un PVC para que el almacenamiento sobreviva a los reinicios del Pod.

**Keycloak**

Lo desplegamos a través del operador oficial de keycloak, porque nos gestiona toda la compejidad que tiene su despliegue y configuración. Con este operador, en lugar de crear nosotros todos los recursos de Kubernetes para Keycloak (Deployment, Service, Ingress, ConfigMap, Secret...), le decimos qué queremos en un YAML sencillo y él se encarga del resto.

**Backend Y Frontend**

Una vez la imagen está en GHCR, el despliegue sigue el mismo patrón: ficheros YAML aplicados con kubectl apply -f “ fichero.yaml”.

El despliegue se realiza aplicando los manifiestos YAML. Todos los comandos se ejecutan desde la carpeta deployment/. El orden de despliegue (siempre se despliega primero aquello de lo que otros dependen, y después lo que depende de ello) es:

1. Namespaces (postgres, keycloak, apps)

microk8s kubectl apply -f production/namespaces/

2. PostgreSQL (ConfigMap + PVC + Deployment/Service)

   microk8s kubectl apply -f production/infrastructure/postgres/

3. Keycloak Operator (CRDs + Operator)

   Instalamos el programa operador:

   microk8s kubectl apply -n keycloak -f

   production/infrastructure/keycloak/operator/operator_26.5.5.yaml

   Aplicamos los CRDs( le dicen a kubernetes quien es keycloak):

   microk8s kubectl apply -f production/infrastructure/keycloak/crds/

   Creamos nuestra instancia de keycloak:

   microk8s kubectl apply -f production/infrastructure/keycloak/deployment/

4. Backend (GHCR Secret + ConfigMap + Deployment/Service)

   Crear el Secret con las credenciales de GHCR:

   bash production/confidential/create-ghcr-secret.sh

   Desplegar el backend:

   microk8s kubectl apply -f production/apps/backend/

5. Frontend (ConfigMap + Deployment/Service + Ingress)

   microk8s kubectl apply -f production/apps/frontend/

**CONFIGURAR etc/hosts**

Los dominios (auth.maryshopping.com, maryshopping.com) no existen en Internet, son nombres inventados para el proyecto. El navegador y el backend necesitan saber a que IP conectarse, y como no hay DNS real que los resuelva, se usa este fichero (etc/hosts) como “como DNS local” que se consulta antes que internet. Debemos conocer la IP en la que microk8s está escuchando peticiones ( comando para obtener la IP: kubectl get nodes -o wide). El ingress de microk8s expone esos servicios directamente en esa IP, de tal manera que todo el tráfico de esos dominios llega al clúster y el Ingress lo distribuye:

maryshopping.com/ → maryshopping-frontend:3000

maryshopping.com/maryshopping → maryshopping-backend:8089

auth.maryshopping.com/ → keycloak-service:8080

Editamos el archivo añadiendo al final del fichero la IP seguida de las rutas DNS.

## CONFIGURACIÓN DE LA APLICACIÓN

Kubernetes utiliza el patron "configuración externalizada" (separar el código de la configuración). La imagen Docker, solo contiene el código, la configuración llega desde fuera (configMap). Cuando arranca kubernetes, éste inyecta ese configMap en el contenedor y así ya tiene la configuración específica para ese entorno específico. Esto permite cambiar la configuración sin reconstruir la imagen Docker. Este patrón (externalizar la configuración) es una de las prácticas fundamentales de las aplicaciones cloud-native.

### Backend

En Kubernetes, la configuración se externaliza en un ConfigMap (deployment/production/apps/backend/backend_configmap.yaml) que sobreescribe el application.yaml.

### Frontend

La configuración del frontend en desarrollo está en: frontend/nuxt.config.ts

La variable más importante es la URL base del backend: apiBase. En Kubernetes, esta variable se sobreescribe mediante un ConfigMap (deployment/production/apps/frontend/frontend_configmap.yaml) que inyecta la variable de entorno NUXT_PUBLIC_API_BASE apuntando al backend interno.

### Keycloak

Igualmente, la URL de Keycloak (para el login desde el frontend) se externaliza como variable de entorno:

NUXT_PUBLIC_KEYCLOAK_URL: http://auth.maryshopping.com

## PostgreSQL

Es un programa que no escribes tú (software de terceros) . Los programas externos en contenedores Docker se configuran a través de variables de entorno, usando las variables que la imagen oficial de postgreSQL espera (POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, PGDATA). Estas variables se las pasamos desde kubernetes cargándolas desde un configMap y referenciandolo en el Deployment en envFrom.

## ACCESO A LA APLICACIÓN

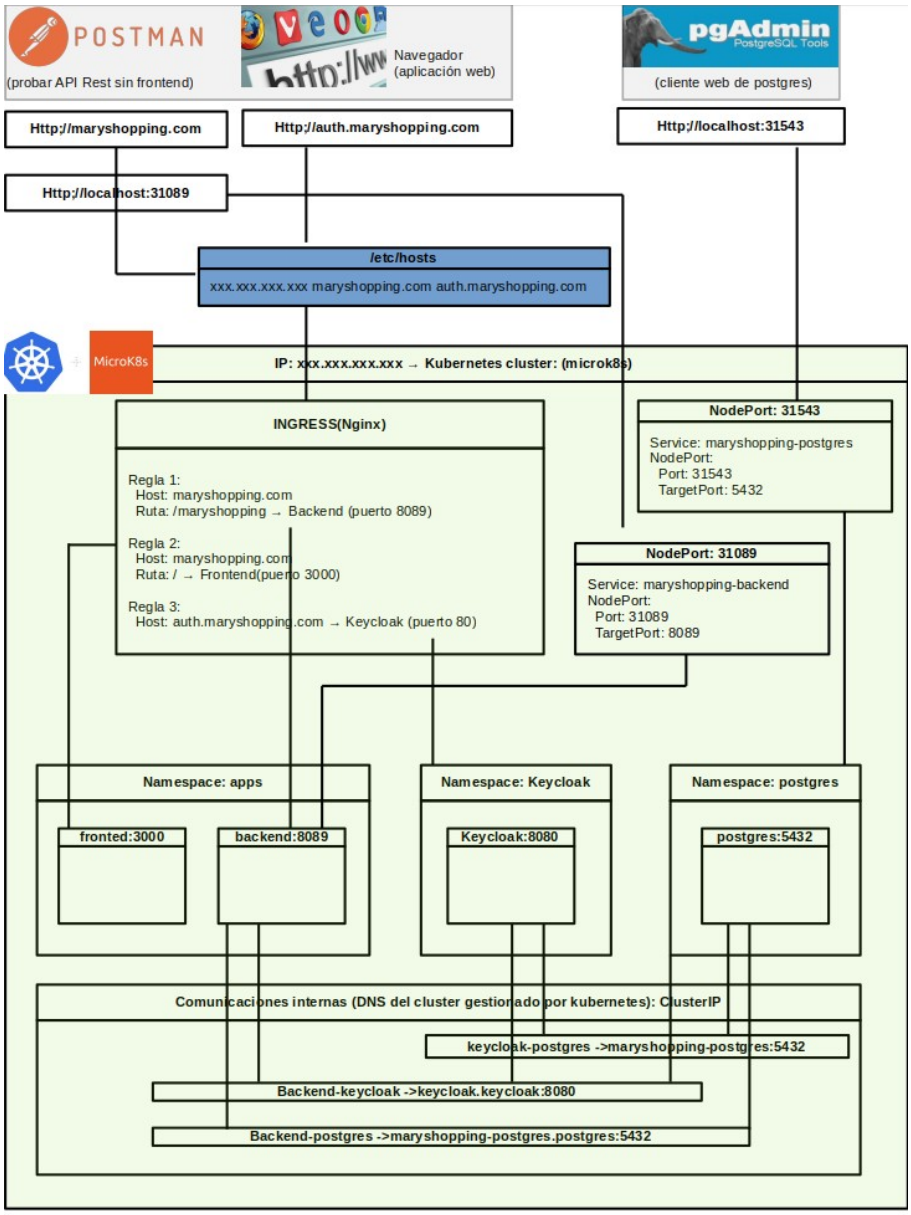



**Figura 18: Mapa de acceso a la aplicación.**

# 3. Conclusiones y recomendaciones

## CONCLUSIONES PERSONALES

Este proyecto ha sido una experiencia de aprendizaje muy enriquecedora. Cuando empecé a investigar sobre Domain-Driven Design, Vertical Slice Architecture y Monolito Modular, muchos de los conceptos me resultaban abstractos y difíciles de entender. Sin embargo, a medida que los iba aplicando en código real, fui comprendiendo por qué cada decisión tiene sentido.

La principal conclusión es que la arquitectura importa. No es algo que solo deba preocupar a los grandes proyectos. Incluso en un proyecto académico como este, tener una arquitectura clara y consistente ha hecho que el código sea más fácil de leer, de modificar y de explicar.

## ALCANCE DE OBJETIVOS

Se han alcanzado todos los objetivos planteados en el apartado 1.2 Objetivos.

API REST funcional con CRUD completo de catalog, customers y orders. Módulo de reporting (reports) con consultas agregadas. Arquitectura DDD + Vertical Slice implementada de forma consistente. Módulos con esquemas de base de datos independientes. Configuración Flyway multi-módulo personalizada. Despliegue con Docker (desarrollo) y Kubernetes/MicroK8s (producción). Pruebas manuales con Postman. Documentación académica completa. Frontend desarrollado con Nuxt 4 (Vue 3.5), Vuetify 4, Pinia y TypeScript. Autenticación JWT integrada con Keycloak. Aplicación práctica de principios cloud-native. Programación declarativa y modularización de la infraestructura.

## MEJORAS FUTURAS

Añadir tests automatizados (mejoraría la confianza en el código), documentar la API con OpenAPI/Swagger (útil para quien quiera consumir la API), automatizar la construcción de la imagen Docker y su publicación en GHCR cada vez que se hace un push al repositorio ( CI/CD), utilizarHTTPS ( para una versión segura) y mejorar la arquitectura del frontend.

# 4. Bibliografía y webgrafía

**Bibliografía**

Baldwin, C. Y., & Clark, K. B. (2000). Design Rules, Volume 1: The Power of Modularity. MIT Press.

Brooks, F. P. (1975). The Mythical Man-Month: Essays on Software Engineering. Addison-Wesley.

Conway, M. E. (1968). How Do Committees Invent? Datamation, 14(5), 28-31.

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.

Ferraiolo, D. F., & Kuhn, D. R. (1992). Role-Based Access Controls. Proceedings of the 15th National Computer Security Conference, pp. 554-563.

Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. IT Revolution.

Goetz, B. (2022). Data-Oriented Programming in Java. InfoQ. https://www.infoq.com/articles/data-oriented-programming-java/

Heinemeier Hansson, D. (2016). The Majestic Monolith. Signal v. Noise (Basecamp).

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc7519

Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Data Systems. O'Reilly Media.

Kniberg, H., & Ivarsson, A. (2012). Scaling Agile @ Spotify with Tribes, Squads, Chapters & Guilds.

Leach, P. J., Mealling, M., & Salz, R. (2005). A Universally Unique IDentifier (UUID) URN Namespace. RFC 4122. IETF. https://datatracker.ietf.org/doc/html/rfc4122

MacCormack, A., Rusnak, J., & Baldwin, C. Y. (2006). Exploring the Structure of Complex Software Designs: An Empirical Study of Open Source and

Proprietary Code. Management Science, 52(7), 1015-1030. Harvard Business School / MIT.

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.

Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM, 15(12), 1053-1058. https://doi.org/10.1145/361598.361623

Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning Publications.

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-Based Access Control Models. IEEE Computer, 29(2), 38-47. https://doi.org/10.1109/2.485845

Skelton, M., & Pais, M. (2019). Team Topologies: Organizing Business and Technology Teams for Fast Flow. IT Revolution.

Vernon, V. (2013). Implementing Domain-Driven Design. Addison-Wesley Professional.

Young, G. (2010). CQRS Documents. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

## WEBGRAFÍA

CNCF — Cloud Native Definition. (abril 2026). Cloud Native Computing Foundation. https://github.com/cncf/toc/blob/main/DEFINITION.md

Spring Boot Reference Documentation. (abril 2026). VMware (Broadcom). https://spring.io/projects/spring-boot

Spring Data JPA Reference Documentation. (abril 2026). VMware (Broadcom). https://spring.io/projects/spring-data-jpa

Spring Data JPA — Query Methods. https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html

Spring Data JPA — Projections. https://docs.spring.io/spring-data/jpa/reference/repositories/projections.html

Spring Data JPA — Custom Repository Implementations. https://docs.spring.io/spring-data/jpa/reference/repositories/custom-implementations.html

Spring Modulith Reference Documentation. (abril 2026). VMware (Broadcom). https://docs.spring.io/spring-modulith/reference/

Flyway Documentation. (abril 2026). Redgate. https://documentation.red-gate.com/flyway

Flyway — Configuration. https://documentation.red-gate.com/flyway/flyway-concepts/flyway-projects/configuration

Flyway — Migrations. https://documentation.red-gate.com/flyway/flyway-concepts/migrations

Spring Boot Reference — Flyway Properties. https://docs.spring.io/spring-boot/reference/data/sql.html#data.sql.schema-initialization.flyway

Stack Overflow — “Spring Boot multiple Flyway schemas.” https://stackoverflow.com/questions/44576956/configure-flyway-to-run-multiple-schemas

Baeldung — “Database Migrations with Flyway.” https://www.baeldung.com/database-migrations-with-flyway

PostgreSQL Documentation. (abril 2026). The PostgreSQL Global Development Group. https://www.postgresql.org/docs/

Project Lombok. (abril 2026). The Project Lombok Authors. https://projectlombok.org/

Docker Documentation. (abril 2026). Docker Inc. https://docs.docker.com/

Fowler, M. Shared Database. (abril 2026). martinfowler.com. https://martinfowler.com/bliki/SharedDatabase.html

Fowler, M. CQRS. (abril 2026). martinfowler.com. https://martinfowler.com/bliki/CQRS.html

Fowler, M. Bounded Context. (abril 2026). martinfowler.com. https://martinfowler.com/bliki/BoundedContext.html

Fowler, M. Domain-Driven Design. (abril 2026). martinfowler.com. https://martinfowler.com/tags/domain%20driven%20design.html

Fowler, M. Polyglot Persistence. (abril 2026). martinfowler.com. https://martinfowler.com/bliki/PolyglotPersistence.html

Fowler, M. Reporting Database. (abril 2026). martinfowler.com. https://martinfowler.com/bliki/ReportingDatabase.html

Google Java Style Guide. (abril 2026). Google. https://google.github.io/styleguide/javaguide.html

Java Language Specification, §6.1 — Declarations. (abril 2026). Oracle. https://docs.oracle.com/javase/specs/jls/se25/html/jls-6.html

OpenJDK — Java 25 Release Notes. (abril 2026). openjdk.org. https://openjdk.org/projects/jdk/25/

JEP 395 — Records. openjdk.org. https://openjdk.org/jeps/395

JEP 444 — Virtual Threads. openjdk.org. https://openjdk.org/jeps/444

JEP 441 — Pattern Matching for switch. openjdk.org. https://openjdk.org/jeps/441

Twelve-Factor App. (abril 2026). https://12factor.net/

RFC 4122 — A Universally Unique IDentifier (UUID) URN Namespace. https://datatracker.ietf.org/doc/html/rfc4122

OWASP — Insecure Direct Object References (IDOR). https://owasp.org/www-project-web-security-testing-guide/

Unix time. Wikipedia. https://en.wikipedia.org/wiki/Unix_time

MDN Web Docs — Date.now(). Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/now

Stack Overflow Developer Survey 2024. https://survey.stackoverflow.co/2024/

JetBrains — The State of Developer Ecosystem 2024. https://www.jetbrains.com/lp/devecosystem-2024/

State of JavaScript 2024. https://stateofjs.com/

Vue.js Documentation. (abril 2026). https://vuejs.org/

Nuxt Documentation. (abril 2026). https://nuxt.com/

Vuetify 4 Documentation. (abril 2026). https://vuetifyjs.com/

Pinia Documentation. (abril 2026). https://pinia.vuejs.org/

Axios Documentation. (abril 2026). https://axios-http.com/

Kubernetes Documentation. (abril 2026). https://kubernetes.io/docs/

MicroK8s Documentation. (abril 2026). Canonical. https://microk8s.io/docs

Kubernetes — Concepts: Service (NodePort). (abril 2026). https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport

Keycloak Documentation. (abril 2026). Cloud Native Computing Foundation. https://www.keycloak.org/documentation

Keycloak Operator — Installation Guide. (abril 2026). Keycloak. https://www.keycloak.org/operator/installation

Keycloak Operator — Basic Deployment. (abril 2026). Keycloak. https://www.keycloak.org/operator/basic-deployment

GitHub Container Registry (GHCR) Documentation. (abril 2026). GitHub. https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry

Kubernetes — Concepts: Operator Pattern. (abril 2026). https://kubernetes.io/docs/concepts/extend-kubernetes/operator/

Kubernetes — Custom Resource Definitions (CRDs). (abril 2026). https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/

Spring Boot — Externalized Configuration. (abril 2026). VMware (Broadcom). https://docs.spring.io/spring-boot/reference/features/external-config.html

Postman Documentation. (abril 2026). Postman Inc. https://learning.postman.com/docs/

Maven Documentation. (abril 2026). The Apache Software Foundation. https://maven.apache.org/guides/

OWASP Authentication Cheat Sheet. (abril 2026). OWASP Foundation. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP JWT Security Cheat Sheet. (abril 2026). OWASP Foundation. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

Spring Security Reference Documentation. (abril 2026). VMware (Broadcom). https://docs.spring.io/spring-security/reference/